

Optimising retrieval performance in RAG systems: A new growing window semantic chunking strategy to address weak semantic boundaries

Antonio Moreno-Cediel, Eva Garcia-Lopez , Antonio Garcia-Cabot *, David De-Fitero-Dominguez

Departamento de Ciencias de la Computación, Universidad de Alcalá, Alcalá de Henares, Madrid 28801, Spain

ARTICLE INFO

Keywords:

Artificial intelligence
NLP
RAG
Semantic splitting
Sentence textual similarity
Chunking

ABSTRACT

The release of ChatGPT in November 2022 signified a pivotal shift within the domain of Natural Language Processing, particularly in the context of chatbot technology. Since then, chatbots powered by Large Language Models have been widely adopted, demonstrating remarkable capabilities across a wide range of tasks. In light of the substantial impact of this technology, the study of its application in specific domains has emerged as a crucial area of research. However, given that these chatbots are based on Large Language Models, they are subject to the well-known limitations of this technology when handling with knowledge-intensive tasks, and are prone to generate hallucinated answers. The RAG (Retrieval-Augmented Generation) architecture, which comprises an indexing phase, was developed to address this issue. During the indexing phase, text should be split into different chunks using a chunking technique. We thoroughly analyse state-of-the-art text chunking techniques used in RAG pipelines and propose a novel semantic text chunking technique. To evaluate the effectiveness of our technique within RAG pipelines, an exhaustive evaluation framework has been defined and applied. This evaluation also enables the comparison with previous techniques, resulting in a noticeable improvement in respect to state-of-the-art strategies.

1. Introduction

The development of chatbots has been closely intertwined with the progression of Natural Language Processing (NLP) technologies. However, their actual growth is directly aligned with the emergence of Large Language Models (LLMs) based on the Transformer architecture [1]. Since its introduction in 2017, the Transformer architecture has consistently achieved state of the art results across a range of NLP tasks, including text generation, machine translation, or question answering [1–3]. Consequently, contemporary chatbot implementations leverage this architecture.

Despite the continuous enhancement of LLM performance, these models suffer from hallucinations when employed in knowledge-intensive tasks, defined by Lewis et al. [4] as those tasks in which humans could not reasonably be expected to perform without access to an external knowledge source. Hallucinations manifest as LLM-generated outputs that, while seeming plausible, deviate from the user's input, previously generated context or factual knowledge, thereby compromising the reliability of LLMs in real-world scenarios [5]. This

drawback restricts the use of chatbots in contexts where, despite their usability, the need of a robust and reliable answer is crucial. For example, while chatbots can facilitate the process of retrieving information from extensive documentation, such as aircraft repair manuals, the critical nature of these tasks necessitates the assurance of the accuracy of the information, as well as the exclusion of inaccurate or fabricated information.

To mitigate LLM hallucinations, Lewis et al. [4] introduced the Retrieval Augmented Generation (RAG) architecture in 2020. The RAG architecture enhances the LLM's parametric memory, which is inherently limited to the knowledge acquired during training, by integrating a non-parametric memory component: a retriever. This retriever searches and retrieves relevant information from a vector index in response to user queries [4]. Therefore, the performance of the RAG architecture is completely dependent on the quality of the retrieved information and its relationship with the user query, as the retrieval of irrelevant data may lead to incorrect responses [6,7].

Given the critical role of Information Retrieval (IR) in determining overall RAG performance, optimizing IR system efficacy is paramount.

* Corresponding author.

E-mail address: a.garciac@uah.es (A. Garcia-Cabot).

<https://doi.org/10.1016/j.knosys.2025.114896>

Received 30 April 2025; Received in revised form 9 October 2025; Accepted 10 November 2025

Available online 11 November 2025

0950-7051/© 2025 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

Despite the extensive research that has been carried out in the field of IR, there are still significant opportunities for improvement, particularly with regard to the text splitting, or “chunking”, process that is undertaken prior to vector index construction. Current research increasingly underscores the importance of this seemingly straight forward step [8]. For instance, a shift away from static chunking approaches toward semantical chunking is gaining momentum since Greg Kamradt introduced it [9].

Semantic chunking is a sophisticated text segmentation technique that leverages the semantic properties of language through the use of text embeddings. Unlike traditional approaches, semantic chunking aims to preserve the coherence of meaning within each chunk by grouping together semantically associated units of text [10]. This method initiates by transforming the input text into a series of vector embeddings using a pre-trained language model, thereby mapping each textual segment to its corresponding vector representation. Chunk boundaries are then determined by evaluating the cosine similarity between consecutive vectors, creating a chunk when the similarity falls below a predetermined threshold.

This paper addresses limitations identified in Kamradt’s semantic splitting approach, presents a method designed to overcome these deficiencies, and proposes a novel evaluation framework for IR systems within RAG architectures. In summary, this paper presents four major contributions.

- A novel approach to semantic text chunking, building upon Kamradt’s technique, demonstrating enhanced retrieval performance according to our results.
- An in-depth study into the evaluation of text chunking impact on retrieval efficiency. This includes the development of a mapping technique to adapt labelled passages from the MIRACL IR dataset [11] for evaluating retrieval performance using custom passages (chunks), and the use of synthetically generated questions to verify the relevance of the retrieved chunks.
- The introduction of four new IR metrics specifically designed for evaluating retrieval performance within the MIRACL dataset when using custom passages. The comprehensive results for all metrics employed in this study – including these newly proposed ones – are publicly available to facilitate benchmarking and comparative analysis in future research.
- A benchmark presenting results across all evaluation metrics outlined in the evaluation strategy, providing a comprehensive assessment of the proposed approach.

The remainder of this paper is structured in the following way: [Section 2](#) reviews the context related to IR and text segmentation techniques used for RAG architectures. [Section 3](#) explains the methodology implemented to enhance the semantical chunking strategy proposed by Kamradt. [Section 4](#) includes the results obtained and [Section 5](#) shows the discussion. Finally, [Section 6](#) presents the conclusions, limitations and future work.

2. Related work

2.1. Information retrieval

Information Retrieval comprises all the activities concerned with the organization, processing, and access to information of all forms and formats. The term ‘Information Retrieval’ was first coined in 1952, and with the advent of computers and the World Wide Web, it gained momentum. Indeed, nowadays IR systems play a very important role in our daily lives, being the core of numerous activities, including web searches [12].

The primary function of an IR system is to match user queries with relevant information it has available. To facilitate this matching process, information is typically analysed and represented in such a way that it

becomes convenient for matching through a technique known as indexing [13]. Throughout the evolution of IR, various indexing approaches have been employed. The most primitive retrieval approaches represented information by manually assigning keywords or phrases, known as index terms, to the pieces of information stored in the system [14]. Later, automated indexing, defined by Salton [15] as the assignment of content identifiers with the aid of computers, was proposed as a solution to address the identified inconsistencies and expedite this process.

The foundation of automated indexing lies on Luhn’s theory, which posits that the frequency of terms within a document correlates with their significance in denoting the document’s subject matter [16]. This led to the widespread adoption of word frequency metrics for indexing, including term frequency (TF), inverse term frequency (IDF), and its composite (TF-IDF) [17,18]. Subsequently, the BM25 [19] algorithm was proposed as a refinement of these measures, addressing the issue of linearly increasing scores with the number of matched relevant terms.

These initial approaches primarily focused on full document retrieval, extracting key terms from entire documents and returning complete documents in response to a queries. This represents a document-level granularity, wherein the smallest retrievable unit is a complete document [17,18,20]. Nevertheless, as Salton and Buckley [21] state, this approach does not fully meet the needs for accurately representing the content of texts. On the other hand, knowledge-based approaches were developed to enhance document representation, incorporating semantic specifications of words in the form of thesauruses and dictionaries [20]. However, the scalability of this methodology was limited by the necessity of diverse knowledge structures across varying contexts [21]. As a result, the division of documents into smaller units of text, such as sections, paragraphs, or even sentences, emerged as a method for a more nuanced content representation [21,22].

Despite the fact that these statistical and knowledge-based approaches lay the foundation stone of indexing, a different course of action was needed to tackle the inherent ambiguities in natural language and capture semantic meaning [21]. Hopefully, with the advent of neural networks and Deep Learning, research focused on semantically meaningful computational text representations has experienced a golden era.

2.2. Automated text representation

The development of Deep Learning models for the automated acquisition of text representations is the culmination of years of in-depth research. The initial applications of deep neural networks focused on generating static word representations, as exemplified by Word2Vec [23] and GloVe [24]. While these approaches have been able to represent words through vectorial word representations (*embeddings*), they do not consider the context surrounding the represented word. Therefore, homonym words are assigned identical vector representations, necessitating the development of “contextualized” word embeddings. Unlike static word embeddings, contextualized embeddings represent words within their surrounding context, being able to provide different embeddings for the same word if its surrounding context differs [25]. The early contributions to this field include Context2Vec [26] and ElMo [27], but the main breakthrough occurred with the advent of the Transformer architecture [1].

The attention mechanism, proposed as the core of the Transformer architecture, endows these models with considerable capacity for encoding text sequences. Based on this architecture, the BERT Transformer model introduced Masked Language Modelling as a novel training objective, designed to capture both left and right contextual information for each word [28]. While not its primary intention, BERT’s strong encoding capabilities led to state-of-the-art performance on Semantic Textual Similarity (STS) tasks. However, due to its high computing requirements, large-scale semantic similarity comparisons, clustering and information retrieval via semantic search were not

applicable for BERT [29]. Thus, Sentence-BERT (SBERT) [29], a refinement of the BERT network that incorporates Siamese and triplet networks, was proposed. SBERT significantly reduced the computational inference time required to obtain semantically meaningful sentence embeddings. This success spurred the development of a new type of Transformer models known as Sentence-Transformers.

Concurrently with the development of Sentence Transformers models, limitations of Large Language Models in knowledge-intensive tasks became apparent. Despite training on extensive datasets, LLMs exhibit demonstrable knowledge limitations [5], stemming from two primary factors: their inherent limitations in fully assimilating the vast array of information contained within their training corpora, and the presence of knowledge gaps in publicly accessible corpora, such as those used by state-of-the-art models like Llama 3 [30]. These limitations often result in a lack of specificity essential for specialized domains. This limitation is particularly pronounced when applying LLMs to knowledge-intensive tasks in fields such as medicine [31], engineering [32] or law [33].

Furthermore, LLMs demonstrate difficulty integrating novel information, frequently relying on pre-existing parametric knowledge rather than newly acquired data. Therefore, attempts to augment LLMs with knowledge through supervised fine-tuning may inadvertently increase the probability of hallucination [34].

As a result, there was a latent need to address the hallucinations produced by LLMs when handling queries that extend beyond the scope of their training data or concern contemporary information [5]. Retrieval Augmented Generation rises as a novel architecture to overcome these challenges by decoupling the parametric memory of the model from external or up-to-date knowledge [4,35].

2.3. Retrieval augmented generation

RAG effectively integrates the strengths of both Information Retrieval and Large Language Models to mitigate the problem of hallucination. It achieves this by leveraging the remarkable performance of embedding-based IR and the generative capabilities of LLMs to produce grounded responses. Even though its first steps were taken through the incorporation of knowledge via PreTraining Models (PTM), the arrival of ChatGPT served to underscore the potential of In-Context Learning (ICL) within LLMs [36]. Hence, RAG research shifted towards optimizing the provision of relevant information to LLMs to answer knowledge-intensive tasks during inference [35]. Therefore, RAG combines Information Retrieval strategies with the generation of text with LLMs using ICL.

2.3.1. Information granularity level in RAG pipelines

In the RAG architecture, defining an appropriate strategy for processing available information is critical [35]. In fact, this includes a core and tricky decision: the granularity level of information [37]. Such is the importance of this process that the relevance of the information retrieved from the IR system inside the RAG architecture is conditioned by this decision [35]. Two main factors influence this judgment. On the one hand, the performance of Sentence Transformer models is limited up to a certain number of tokens. The larger the text, the more difficult it is to obtain a fixed size vector that comprises its semantic meaning. On the other hand, the LLM used during generation also has a pre-defined context length, and exceeding this limit will result in a poor performance of the model [38]. Hence, determining an appropriate granularity level is not trivial, but significantly impacts the overall performance of the entire RAG system [39,40].

Attending the Sentence Transformer limited sequence length, a range of diverse lengths can be found. The Massive Text Embedding Benchmark (MTEB) [41] measures the performance of embedding models on diverse embeddings tasks. Focusing on the STS task, different sequence lengths of models that are oriented towards STS in this benchmark coexist. On the one hand, models with ‘relatively’ large input sequence

lengths are found, such as *jina-embeddings-v3* [42], which accepts inputs up to 8192 tokens. On the other hand, other valuable models such as *NV-Embed-v2* [43], *stella_en_1.5B_v5* [44] or *bilingual-embedding-large* [45] (currently the best of the MTEB benchmark in STS), have more strict input sequence constraints of 4096 and 512 tokens, respectively.

Regarding LLM context limitations, state-of-the-art research endeavours have focused on the development of models with bigger contexts windows. For instance, the largest Llama 3 model (405B parameters) holds a context window of 128 K tokens [30]. Nonetheless, even though higher context sizes enable greater flexibility to ICL, the computational cost of the attention layers of the model will be higher. As a result, using larger context lengths will result in the need of higher GPU memory requirements and longer inference times [46,47]. Additionally, as Shi et al. state [6], adding irrelevant context to LLMs might distract the model, thereby increasing the risk of the model producing hallucinated responses.

Consequently, the granularity applied when using Transformer models for both obtaining contextualized embeddings and generating text tends to be smaller than the ones used with frequency-based approaches. For instance, REALM [48] used the BERT Transformer to encode text, adopting a passage-level granularity instead of processing entire articles. Similarly, Lewis et al. [4] divided each article into disjoint 100-word chunks when introducing the RAG architecture. More recent studies, like RETRO [49] or RPT [50], have explored the use of even smaller retrieval units.

Evidence of this evolution is demonstrated by the wide variation in retrieval granularities over time. Nowadays, as a general conclusion, there is a discernible trend towards the disaggregation of information into smaller units. However, the question remains: to what extent should this disaggregation be pursued? On the one hand, larger chunks can capture more context, but this is often accompanied by an increase in noise, the induction of distractions [6], and a requirement for a longer processing time, as well as higher costs [46,47]. Conversely, smaller chunks may not fully include the necessary context, but they have less noise and are easier to process [35].

2.3.2. Text chunking strategies

With the rise of RAG-based chatbots, frameworks like Langchain [51] or LlamaIndex [52] have been developed to implement several text-splitting strategies. The most common method is to split the document into chunks on a fixed number of tokens [35]. The Langchain Base Text Splitter [53] is representative of this method. Besides, there are a little more advanced chunking strategies, such as character splitter and recursive character splitter. The Character Splitter [54] is an alternative that functions by splitting texts according to user-defined character separators or a maximum chunk length. In contrast, the Recursive Character Splitter [55] undertakes an iterative process to split the text based on a sequence of characters, prioritizing those that yield chunks within the specified chunk length. In the event that the process of splitting fails to achieve the desired outcome by employing the characters that have been provided, the maximum chunk size is then applied. Even though these two strategies provide different chunk sizes, they both remain constrained by a maximum chunk size that does not attend semantical reasons.

Knowing the importance of splitting the text into semantically meaningful chunks, specialized splitters have been developed for specific document formats, including HTML [56], Markdown [57] and programming languages [58]. These splitters, unlike their predecessors, leverage the inherent structural properties of the text, employing domain-specific separators such as ‘#’ in Markdown or ‘\nfunction’ in JavaScript. Therefore, these structure-aware splitters tend to produce chunks whose contents are probably related, as in the construction of structured text contents tend to be separated with certain keywords. For example, considering that in Markdown different sections are divided using the hash character, the Markdown text splitter can obtain the content of each individual section. Nevertheless, the performance of

these structure-aware splitters is contingent upon the quality of document structure. Effective results require documents that are well-structured, cohesive, coherent, and adhere to established domain-specific conventions. In such cases, these splitters may improve previous splitting strategies. Otherwise, if documents lack consistent structure, employing format-specific splitting conventions will not yield any advantages over previous strategies. Furthermore, even when a well-structured document is used, if its sections are huge, the resulting chunks may exceed the model or computational context length requirements. Therefore, this chunking technique could be regarded as a compromise between static strategies and more advanced techniques.

2.3.2.1. Semantic splitting. As a consequence of the significant challenge of dividing text into meaningful sections, new and more sophisticated chunking techniques are being developed. Recently, Greg Kamradt put forward a new chunking strategy known as *semantic splitting* [9]. This technique considers the semantic meaning of the text, using the cosine distance between the sentence embeddings as the criterion to split the text into chunks. This strategy can be divided into three differentiated phases: (1) Obtention of window embeddings, where a window is defined as a contiguous sequence of sentences of fixed length extracted from the text; (2) Calculation of cosine distances between window embeddings; and (3) Obtention of the text breakpoints based on the 95th percentile of the cosine distances.

The first phase focuses on a sliding window of size $N-2 + 1$, composed of a central sentence and its antecedent and subsequent N sentences. As Fig. 1 illustrates, the window is initially positioned with its central sentence aligned with the first sentence of the text and then iteratively progresses by sliding this window until its central sentence corresponds to the final sentence of the text. In each step, an embedding, named ‘Window Embedding’ in Fig. 1, is calculated representing the sentences included in the window. After the calculation of all window embeddings, the second phase involves the calculation of cosine distances between contiguous embeddings. Finally, in the third phase, the

95th percentile of these distances is computed and subsequently used as a threshold parameter. This threshold defines a cutoff point beyond which distances are considered significantly different from the norm. Therefore, breakpoints are then established in the central sentences of those windows whose embeddings exhibit a higher cosine distance with the previous window embedding than the percentile that has been computed. For instance, in Fig. 1, the cosine distance between ‘Window Embedding 4’ (E.4) and ‘Window Embedding 5’ (E.5) is supposed to be higher than the threshold, since there is a breakpoint in sentence 5. As a result, text is split into chunks in a dynamic way and attending similarity purposes.

Nonetheless, the semantic splitting proposed by Kamradt [9] is constrained by a significant limitation that may considerably diminish its efficacy. To illustrate this drawback, the splitting example shown in Fig. 1 is considered to analyse in detail how the decisions of including sentences into the chunks are taken. As previously described, the third phase involves calculating the 95th percentile of all cosine distances and establishing breakpoints based on this value. This is achieved by comparing each cosine distance (from the second phase) to the percentile value. Each comparison determines whether the next sentence is added into the current chunk or if a new breakpoint, and thus a new chunk, is created. For instance, if $\text{Dist}(E.1 \sim E.2)$ is lower than the percentile, then sentence number two will be added into the same chunk as sentence number one. According to Fig. 1, this distance must have been below the threshold, as no breakpoint is observed for sentence two. Next, $\text{Dist}(E.2 \sim E.3)$ is used for the purpose of determining whether sentence number three should be incorporated or not within the same chunk. According to the breakpoints obtained in Fig. 1, this distance must have been lower, resulting in the inclusion of sentence three into the same previous chunk. Later, following the same criteria, $\text{Dist}(E.3 \sim E.4)$ will determine if sentence number four should also be assigned to this chunk. Again, the absence of a breakpoint for sentence four implies that this distance also fell below the percentile threshold.

However, a subtle yet crucial issue emerges upon considering which

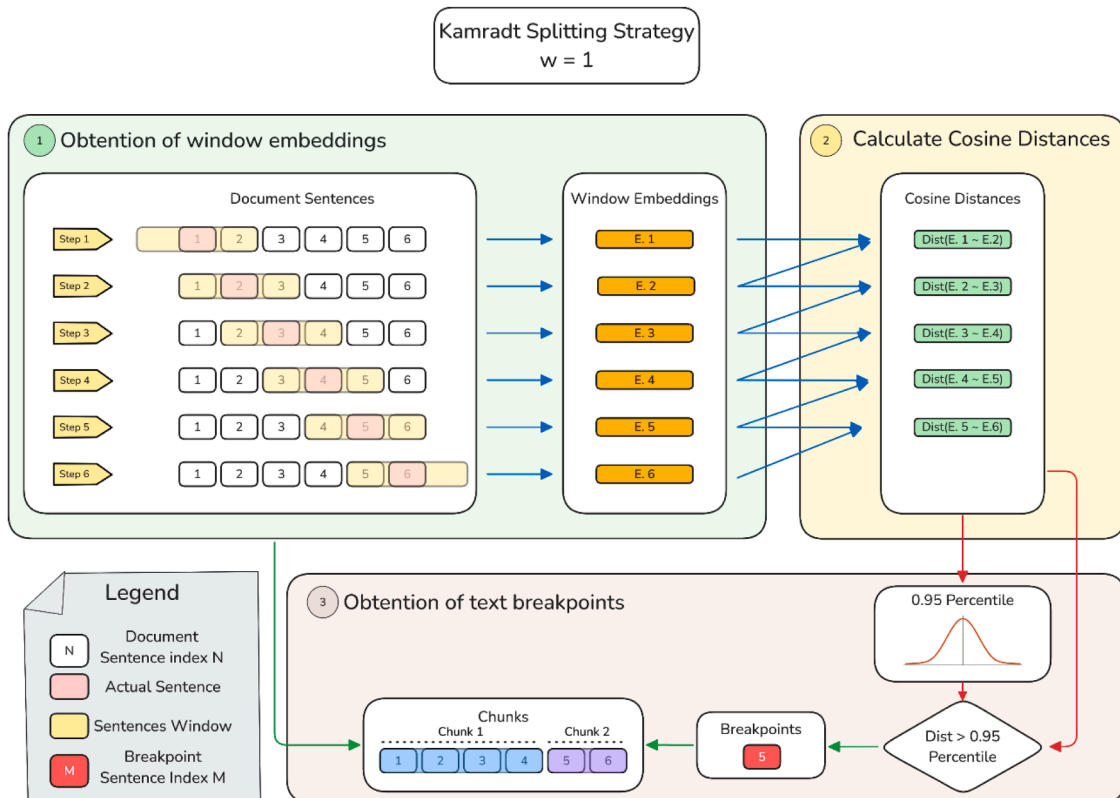


Fig. 1. Kamradt's semantic chunking approach. Using a window size (w) of one sentence, this figure illustrates how Kamradt's strategy split the text.

sentences are represented by the embeddings used in each distance calculation. When evaluating the incorporation of sentence two, embeddings E.1 and E.2 are used, meaning that sentences one, two and three are represented by at least one of the embeddings. With regard to the inclusion of sentence three, embeddings E.2 and E.3 are considered, thereby representing sentences one to four. Finally, with regard to the addition of sentence four, embeddings E.3 and E.4 are employed, which represent sentences two, three, four and five. Consequently, the semantic contribution of sentence one is excluded when deciding whether to include sentence four in the same chunk. Fig. 2 illustrates this behaviour. This casuistry gives rise to the risk of generating non-semantically meaningful chunks, adding sentences into chunks without having previously compared them to the entire semantical meaning of the chunk. Therefore, the creation of longer chunks containing several sentences that vary in meaning is likely to occur.

Nowadays, while fixed-size and recursive splitting approaches continue to predominate, more sophisticated architectures are emerging trying to overcome the previous limitations and improve the retrieval phase. For instance, highlighting the need of a method that dynamically determines the chunking size, Zhong et al. [39] have proposed a mechanism that automatically selects the optimal granularity based on the query formulated by the user. Therefore, text is split into different levels of granularity using a static strategy. When a query is received, a routing strategy is used to select the most appropriate granularity level according to the query using a Multilayer Perceptron. Consequently, although different granularities may be employed, the splitting strategy remains static and may not consider semantic meaning, as previously mentioned. Moreover, related fields such as image retrieval, are increasingly adopting semantic retrieval techniques to enhance retrieval effectiveness through semantic-context enrichment of image data. Nevertheless, these approaches typically do not require text splitting, as each image is commonly associated with short textual descriptions [59, 60].

In addition, recent RAG evaluation studies [61] emphasize the influence of the chosen chunking strategy. However, the evaluation of different strategies is left for future work.

In summary, effective text splitting of text from documents is a critical task within the RAG pipeline. Therefore, ongoing research is dedicated to discovering the most effective splitting techniques to enhance the RAG retrieval phase. To address the limitations of existing methods, a novel semantic splitting technique is proposed in this paper. This technique is based on the Kamradt's original approach and it demonstrates improved performance on standard IR metrics and within a synthetically generated RAG scenario.

3. Methodology

While Kamradt's approach towards dynamic text splitting is a valid strategy, in this study, we present a novel method that has been defined

for addressing its limitations. Considering Kamradt's semantic splitting strategy as our baseline, some variations have been implemented to enhance its performance.

Firstly, to mitigate the potential issue of creating lengthy and non-semantically meaningful chunks, the sliding window mechanism has been modified. Instead of sliding, our approach dynamically increases the window size as new sentences are incorporated. Contrary to Kamradt's approach, which first calculates all the embeddings and distances to obtain the text breakpoints, our method makes the decision to split the text iteratively as it moves the window through the sentences of the text. This process entails the comparison of two embeddings: the first one representing all the sentences included in the actual chunk, and the second one representing the next m sentences under consideration for inclusion. Consequently, new sentences will be incorporated into the current chunk solely if they exhibit semantic similarity to the entire chunk. In this case, semantic similarity will be assessed using cosine similarity between sentence embeddings, and a predefined threshold will be used to determine whether the similarity is sufficient. This threshold ensures that only sentences with a high degree of semantic similarity are grouped together. This modification facilitates the discernment of gradual shifts in meaning, since the embedding represents the cumulative meaning of the entire chunk rather than a limited window of sentences.

Secondly, to improve computational efficiency, a two-stage sentence inclusion strategy is employed. Since the initial sentences of each chunk must be related, the chunk is initially built with the first n sentences and, later, subsequent sentences are included in groups of m sentences, provided they meet semantic similarity requirements.

In light of the aforementioned considerations, the proposed splitting strategy is outlined as follows. Firstly, the text is separated into sentences, thereby obtaining a list of all the sentences in the text. Later, the initial chunk is initialized with the first n sentences. Subsequently, two embeddings are calculated. The first one represents all the sentences included in the chunk, and the second one represents the following m sentences. Once both embeddings have been calculated, the cosine distance between them is obtained, resulting in two possible paths. On the one hand, if the cosine distance is lower than the specified threshold, the m sentences are incorporated into the current chunk. In this case, the same process will be repeated, with the first embedding now encompassing not only the first n sentences but also the recently included m sentences. In other words, the first embedding will consider the entire chunk, while the second embedding will represent the following m sentences. Continuously, this process will add new sentences into the current chunk in groups of m sentences. On the other hand, when the cosine distance between both embeddings is greater than the threshold, the current chunk will be closed with the sentences included until that moment. Then, a new chunk will be initialised including the following n sentences. Later, the same strategy will be followed onwards, including more sentences until the threshold is exceeded.

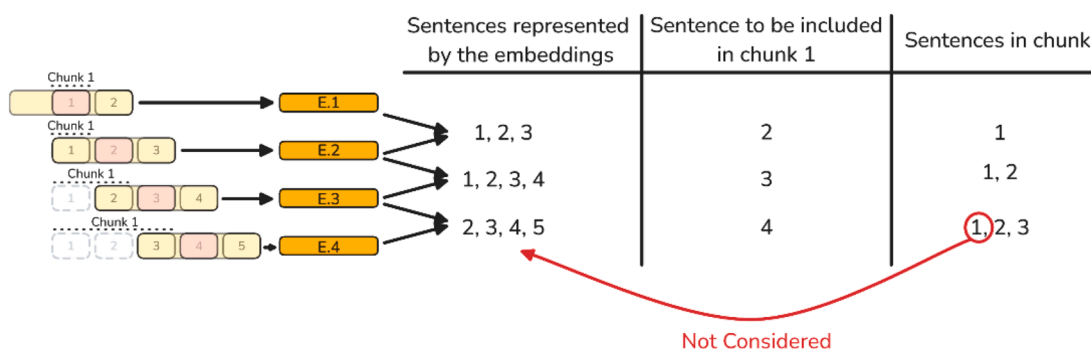


Fig. 2. Kamradt's semantic chunking limitation. As the window moves, the initial sentences are not taken into consideration. Once the central position of the window is located at the third sentence, the first sentence is no longer considered.

Fig. 3 illustrates the proposed strategy with a three-step sample. In the first step, the first n sentences (initial size) are automatically added to the current chunk (chunk 1), as it is initially empty. Then, the embeddings are computed, with one embedding created for the specified n sentences and another created for the subsequent m sentences (regular size). The cosine distance between both is measured and, since in this case it is lower than the threshold, sentence three is added into the current chunk. In the second step, two new embeddings are calculated. The first one considers all the sentences in the current chunk, in this case sentences from one to three, and the second one represents the subsequent m sentences, in this case sentence four. In this example, the cosine distance between the first embedding and the second one is greater than the threshold. Therefore, chunk number one is concluded with sentences ranging from one to three. Finally, in step three, a new current chunk is initialised with the subsequent n sentences, which are sentences four and five. Afterwards, the sixth sentence is included in the current chunk, as its cosine distance between both embeddings is lower than the established threshold. As a result, the text has been split into two chunks: the first one includes the first three sentences, while the second one includes the fourth to sixth sentences.

Please note that, when the next m sentences are not included in the current chunk, the new chunk is initialised with the following n sentences, not with the following m . This strategy assumes that the initial sentences of each chunk are related, giving the variable n a higher value than to the variable m . This approach enhances computational efficiency by reducing the number of required embeddings calculations.

Once defined a splitting strategy that, we hypothesize, signifies an improvement on previous strategies, we focused on using this splitting strategy, evaluating its performance and comparing it to existing baselines. Hence, the following steps are taken: (1) definition of sample texts, (2) application of the splitting strategy across a range of configurations, (3) indexing, and (4) evaluation of the information retrieval performance according to each splitting strategy.

3.1. Definition of sample texts

Looking forward to performing a reliable analysis of the splitting strategy performance, it has been applied to a wide variety of texts across different domains. Specifically, we used the Spanish Wikipedia dump provided by MIRACL, a well-known IR dataset. Using the *WikiExtractor* tool [62], these dumps have been converted into JSON-formatted files of a maximum of 4 MB. Each file contains the article id, title and text, as well as other properties irrelevant to this study. Additionally, as we need to apply the Kamradt's strategy to compare it with our results, articles with fewer than two sentences have been discarded since applying the percentile operation of the Kamradt's approach is not possible in those cases.

As a result, in total, 1572,060 Wikipedia articles have been used in this study. Statistics related to their content can be found in Table 1.

To demonstrate the cross-lingual validity of our approach, a subset of 156,070 articles has been randomly sampled from the English Wikipedia dump provided by MIRACL. Descriptive statistics for this subset are presented in Table 2.

3.2. Application of the splitting strategy

After all the validated MIRACL Wikipedia articles are gathered, the splitting strategy proposed is applied to the text of each article. This process generates two different JSON files, mirroring Kamradt's approach by separating chunks and distances. The first file contains the semantic chunks obtained during the semantic splitting process, along with sentence indices indicating the start and end of each chunk. The second file stores the cosine similarities calculated during the splitting process and includes an embedding property containing the embedding representation of each chunk's text.

During semantic splitting, numerous embeddings are calculated.

While most are transient and only serve for comparison, saving the correct embeddings optimizes future tasks. In Fig. 3, the first chunk is finally defined at the end of the second step, when the cosine distance between embeddings surpasses the threshold. The first embedding of a closed chunk represents all the sentences in that chunk; therefore, its preservation avoids redundant recalculation. As a result, this splitting strategy provides text chunks and their corresponding embeddings concurrently, offering a key efficiency advantage. While Kamradt's approach necessitates a separate embedding calculation for each split chunk, our method avoids this additional computational step. Therefore, despite the generally higher computational demands of semantic splitting strategies, attributable to the use of embedding models, our approach significantly reduces overall computational costs by integrating embedding generation with the splitting process.

As mentioned, our splitting strategy configuration relies on two parameters. First, the number n of sentences that are directly picked at the beginning of each chunk (initial size), and second, the number m of sentences whose addition into the chunk is evaluated based on semantic similarity (regular size). In order to obtaining the best possible configuration, three different configurations have been defined based on the article statistics shown in Table 1. Firstly, the initial size must be determined based on the number of sentences in the articles. As shown in Table 1, articles exhibit a mean of 19.41 sentences and a median of 10.0 sentences. However, the standard deviation (35.92) indicates significant variability in the number of sentences per article. Some articles contain only two sentences, while others exceed two thousand. Therefore, the initial size has been set to four, six, and eight sentences, thus avoiding the creation of chunks comprising entire articles by defining values below the median. Secondly, the regular size is also defined based on sentence statistics. It is essential that this value remain lower than the initial size, since defining a higher value would be illogical. The initial size represents the number of initial sentences presumed to be related because they constitute the beginning of the chunk. Conversely, the regular size signifies the quantity of new sentences to be incorporated into the current chunk. This process necessitates caution to ensure that unrelated sentences are not included. Consequently, the regular size values must be smaller than the initial size values, since there is no certainty that these new sentences are related to the previous ones. As a result, regular sizes are set to two, three, and four. The three configurations are summarized as follows: (1) $n = 4, m = 2$; (2) $n = 6, m = 3$; and (3) $n = 8, m = 4$. The distance threshold has been set to 0.05, looking for high semantical meaningful chunks.

Similarly, Kamradt's splitting strategy has been applied to all the gathered Wikipedia articles. In this case, a window size of three was used, since Kamradt stated that this configuration achieves the best performance [9]. Additionally, the same threshold percentile used by Kamradt has also been set, which is the 95th percentile.

Experiments conducted on a subset of the English Wikipedia articles have extended previous configurations of the Kamradt approach by evaluating window sizes of two, three, and four sentences while maintaining the 95th percentile threshold. The parameters for our approach have remained unchanged. To enable a comparative analysis with non-semantic splitting approaches, we implemented a static splitter that employs line breaks ('\n') as its splitting criterion as a non-semantic baseline.

Regarding the technical details, an embedding model trained for the Spanish language is needed to obtain the dense vectorial representations during the splitting process, since the sample texts are written in Spanish. The *jina-embeddings-v3* [42] model was chosen for this study because it shows high performance in semantic textual similarity and currently ranks third on the MTEB leaderboard [41]. Furthermore, this model is multilingual and has been finetuned for both Spanish and English.

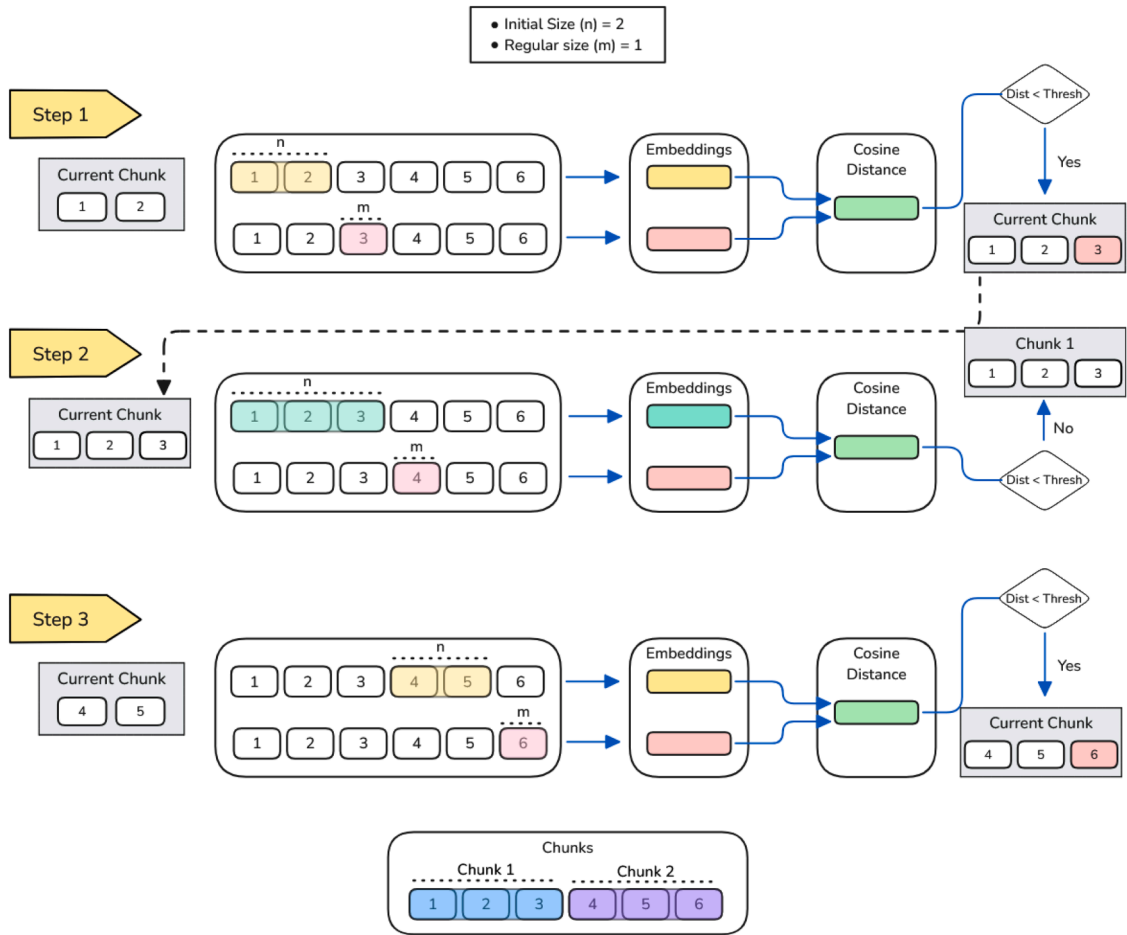


Fig. 3. Semantic chunking strategy proposed. Using an initial size (n) of two sentences and a regular size (m) of one, this figure illustrates how the proposed strategy splits the text.

Table 1

Statistics for validated articles (>1 sentence) from the MIRACL Wikipedia dump in Spanish.

	Mean	Std	Median	Max	Min
Words	484.79	1037.96	205.00	92,298	3
Sentences	19.41	35.92	10.00	2920	2

Table 2

Statistics for a random sample of 156,070 validated articles (>1 sentence) from the MIRACL Wikipedia dump in English.

	Mean	Std	Median	Max	Min
Words	472.68	912.73	198	37,146	3
Sentences	24.68	42.14	12	1657	2

3.3. Indexing

Once the text has been split, its chunks have been inserted into vector databases in the form of embeddings. To this end, we have used Qdrant,¹ a free, locally deployable vector database software, to create four different vector databases for the Spanish data. Each database contains the chunks obtained from one of the four different strategies (three of our configurations plus the Kamradt's one). Similarly, seven vector

databases have been created for the English data, each of them containing chunks from one of the seven different strategies (the three configurations of our approach, the three window sizes employed for Kamradt's approach, and the static splitter configuration).

For obtaining the chunk embeddings, no further operations are needed when using the splitting strategy proposed in this paper because, as aforementioned, the embeddings are saved during the splitting process. On the contrary, chunks gathered using Kamradt's semantic splitting technique must be represented as embeddings. The same embedding model, jina-embeddings-v3, is used for this process.

3.4. Evaluation of the information retrieval performance

Since the main purpose of our research is enhancing the retrieval effectiveness inside a RAG architecture, the evaluation approach followed in this study measures the chunking impact on the retrieval performance.

Two different evaluation approaches have been followed: (1) evaluation through various IR metrics using MIRACL-labelled passages and (2) evaluation using synthetically generated questions by matching retrieved chunks with ground truth chunks from which questions have been extracted.

3.4.1. Evaluation with IR metrics using MIRACL

The first evaluation technique uses the MIRACL dataset. However, since the text passages in MIRACL differ from the chunks obtained using the different splitting strategies in our study, MIRACL labels cannot be used to measure the IR metrics of these chunks. Before explaining how

¹ <https://qdrant.tech/>

this issue has been addressed, it is first necessary to provide some background information about the MIRACL dataset.

The MIRACL dataset comprises queries, each of which is associated with ten different passages that are labelled as either relevant (positive) or non-relevant (negative) to the query. These passages are chunks extracted from Wikipedia articles using a static splitter with double line breaks as separators. Each passage, or chunks, is identified by a *docid*, in the format X#Y, where X represents the article id and Y indicates the sequential number of the passage within that article.

A preliminary evaluation can be conducted based solely on article identifiers. Since each chunk is derived from a known article, given a MIRACL query, we can assess whether the source articles of the retrieved chunks match with those in MIRACL. Following this criterion, two metrics are obtained: (1) same relevant article (SRA@K) and (2) same non-relevant article (SNRA@K). To this end, consider the set of queries Q :

$$Q = \{q_1, q_2, \dots, q_s\}$$

For each query q_i , P_i and N_i , which respectively represent the set of article identifiers labelled as relevant and non-relevant for the query, are defined as follows:

$$P_i = \{p_{i,1}, p_{i,2}, \dots, p_{i,a}\} : 1 \leq i \leq |Q|$$

$$N_i = \{n_{i,1}, n_{i,2}, \dots, n_{i,b}\} : 1 \leq i \leq |Q|$$

$$\text{with } |P_i| + |N_i| = 10 \text{ and } P_i \cap N_i = \emptyset$$

It is important to note that the sum of the positive and negative passages is a constant value of ten for all instances of the MIRACL dataset.

C is defined as the set of article ids from which the retrieved chunks come from:

$$C = \{c_1, c_2, \dots, c_k\}$$

Then, given a query q_i , its SRA@K is defined as follows:

$$\begin{aligned} \text{SRA@K} &= \sum_j^K p_{\text{relevance}_j} \\ p_{\text{relevance}_j} &= \begin{cases} 1 & c_j \in P_i \\ 0 & c_j \notin P_i \end{cases} \end{aligned} \quad (1)$$

Analogously, SNRA@K:

$$\begin{aligned} \text{SNRA@K} &= \sum_j^K n_{\text{relevance}_j} \\ n_{\text{relevance}_j} &= \begin{cases} 1 & c_j \in N_i \\ 0 & c_j \notin N_i \end{cases} \end{aligned} \quad (2)$$

This primitive evaluation gives way to a more exhaustive one. If a retrieved chunk belongs to the same article as the MIRACL, which means that the value $p_{\text{relevance}}$ or $n_{\text{relevance}}$ is one, it is possible to corroborate if this chunk is completely contained in that passage. According to this, two new more metrics stand out: (3) relevant chunk contained (RCC@K) and (4) non-relevant chunk contained (NRCC@K). Consider A_{ASCII} as the set of characters included in the ASCII alphabet:

$$A_{\text{ASCII}} = \{\text{char}(x) \mid x \in \mathbb{N}, 0 \leq x \leq 127\}$$

Both a MIRACL article text and a chunk text are made up of sequences of these characters. Then,

$$\text{chunk}_{\text{text}} = t_1 t_2 \dots t_q, t_r \in A_{\text{ASCII}}, \forall r \in \{1, 2, \dots, q\}$$

$$\text{passage}_{\text{text}} = p_1 p_2 \dots p_m, p_s \in A_{\text{ASCII}}, \forall s \in \{1, 2, \dots, m\}$$

Finally, before defining these metrics, it is necessary to define when

one sequence is included inside another one. In this case, it is defined when the chunk text is included in the MIRACL passage.

$$\begin{aligned} \text{chunk}_{\text{text}} \subseteq \text{passage}_{\text{text}} &\Leftrightarrow \exists s \in \{1, 2, \dots, m - q + 1\}, \forall r \\ &\in \{1, 2, \dots, q\}, \text{passage}_{\text{text}_{s+r-1}} = \text{chunk}_{\text{text}_r} \end{aligned}$$

Therefore, considering the passage text sequence from the MIRACL dataset, the chunk text retrieved from the database after querying it with query q_i , its correspondent P_i and N_i , and the set of retrieved chunks ids C , RCC@K and NRCC@K are defined as follows:

$$\begin{aligned} \text{RCC@K} &= \sum_j^K p_{\text{contained}_j} \\ p_{\text{contained}_j} &= \begin{cases} 1 & p_{\text{relevance}_j} = 1, \text{chunk}_{\text{text}_j} \subseteq \text{article}_{\text{text}_j} \\ 0 & p_{\text{relevance}_j} = 1, \text{chunk}_{\text{text}_j} \not\subseteq \text{article}_{\text{text}_j} \end{cases} \\ \text{NRCC@K} &= \sum_j^K n_{\text{contained}_j} \\ n_{\text{contained}_j} &= \begin{cases} 1 & n_{\text{relevance}_j} = 1, \text{chunk}_{\text{text}_j} \subseteq \text{article}_{\text{text}_j} \\ 0 & n_{\text{relevance}_j} = 1, \text{chunk}_{\text{text}_j} \not\subseteq \text{article}_{\text{text}_j} \end{cases} \end{aligned} \quad (3)$$

$$\quad (4)$$

Besides, the obtention of IR common metrics, such as nDCG@K or Recall@K, necessitates two sets of relevance judgements, often referred to as *qrels* (query relevance sets) [63]. First, the one that contains ground truth relevance judgements, and secondly, the one that comprises the judgements performed during the retrieval process, commonly named *run qrels*. The ground truth *qrels* are provided by the MIRACL dataset, but the *run* ones must be built. Therefore, the *run qrels* must be derived from the retrieved chunks.

However, the obtention of the *run qrels* is not a straightforward process. As aforementioned, the Wikipedia articles have been split using a different strategy than the one used by MIRACL, resulting in different text chunks. As a result, the retrieved chunks differ from the MIRACL passages, making their comparison unreasonable. To address this issue, a mapping strategy is employed to align the retrieved chunks with the MIRACL passages. As shown in Fig. 4, for a given MIRACL sample, with positive and negative passages identified using the X#Y format described earlier, a set of 10 (K) chunks, ordered by score, is retrieved by querying the vector database with the sample query. The retrieved chunks are identified in Fig. 4 using the format X@rank, where X is the Wikipedia article id and rank denotes their position in which they were retrieved from the vector database. Therefore, it is not possible to establish a comparison between the MIRACL *qrels* and the *qrels* obtained from the retrieved chunks. This is due to the fact that the ground truth *qrels* (MIRACL *qrels*) represent MIRACL passages (X#Y), whilst the *run qrels* represent chunks obtained with a different strategy (X@rank). Consequently, as illustrated in the green box (bottom box) of Fig. 4, the mapping operation is applied. For each retrieved chunk, if its corresponding article id is found within one of the passages of the MIRACL sample, it is checked whether the content of the retrieved chunk is literally included in the corresponding MIRACL passage. This procedure is analogous to that employed to obtain the RCC@K or NRCC@K metrics. In such cases where all the chunk text is contained, the chunk retrieved is considered to be the MIRACL passage where it is contained (see how 847,061@7 comes into 847,061#0 in Fig. 4). Conversely, in the cases where all the chunk text is not contained, the chunk retrieved cannot be mapped (see how 1989,407@1 is not mapped into 1989,407#0 in Fig. 4). As a result, a retrieved chunks mapped relevance set is built, where some of the retrieved chunks have been mapped to the X#Y format and the comparison with the MIRACL *qrels* is feasible.

In summary, for building up the *run qrel*, the retrieved chunks are mapped to the MIRACL passages in which they are fully contained. Therefore, both *qrels* are considering MIRACL passages, making the comparison conceivable. Fig. 4 illustrates this process.

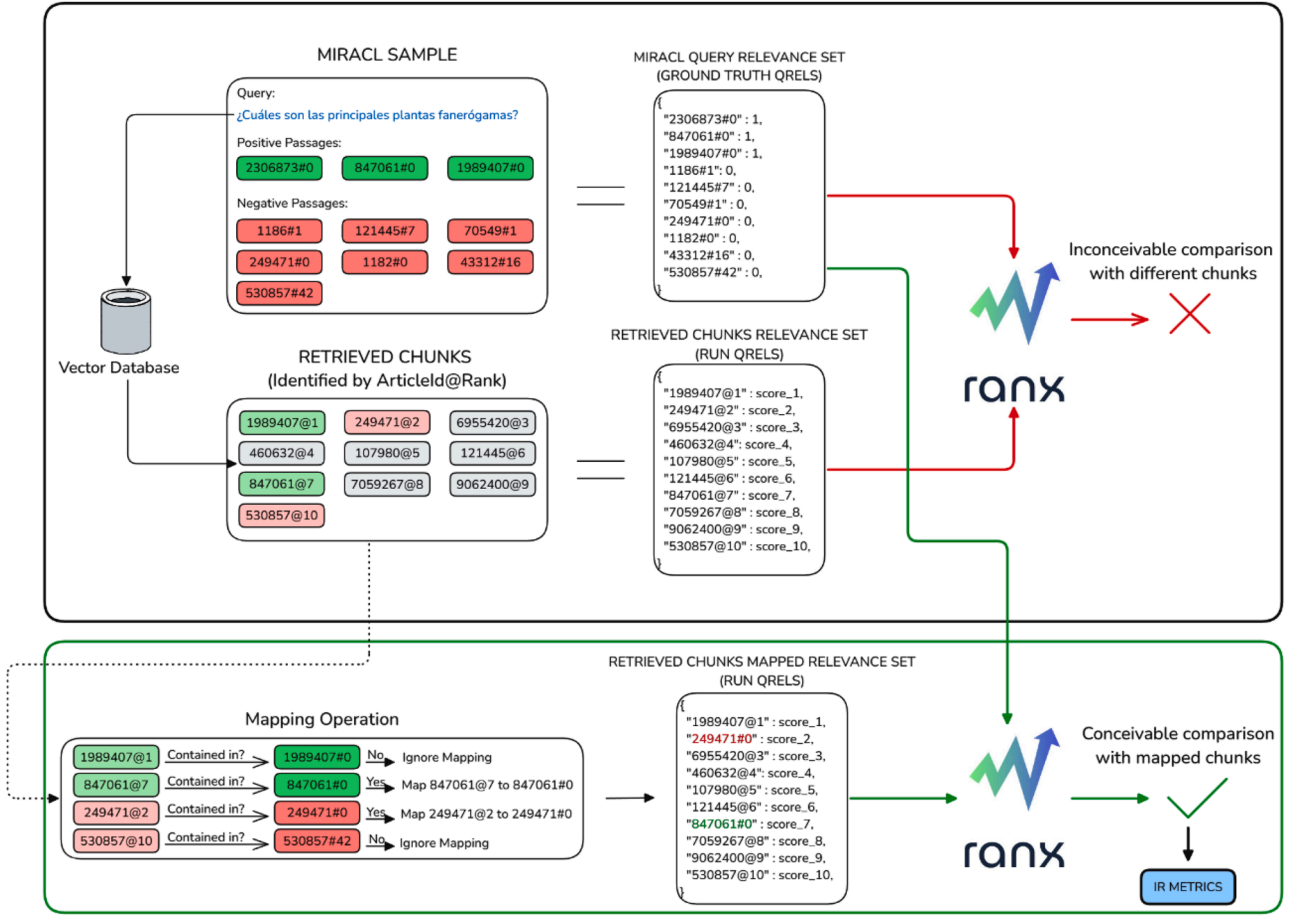


Fig. 4. Qrels mapping operation. The comparison between the MIRACL Qrels and the run Qrels is not conceivable. Therefore, the mapping operation checks if the retrieved chunks are contained in the MIRACL passages.

Thanks to this mapping operation, well-known metrics such as precision, recall or F1, widely applied in the field of artificial intelligence [64], can be computed in their IR variant using the *ranx* python library [65]. A brief description of these metrics is outlined to raise understanding of their meaning and purpose.

- **Hit Rate (hit_rate@K)** computes the fraction of queries for which at least one relevant document is retrieved.
- **Precision** measures the proportion of the retrieved documents that are relevant. Therefore,

$$\text{Precision@K} = r/K \quad (5)$$

Where r is the number of relevant documents retrieved and K is the total number of documents retrieved.

- **Recall** computes the ratio of the number of relevant documents retrieved (r) to the total number of relevant documents (R).

$$\text{Recall@K} = r/R \quad (6)$$

- **F1** represents the harmonic mean of Precision and Recall, balancing both metrics.
- **Mean Reciprocal Rank (MRR)** is an order-aware metric, which, unlike recall@K or precision@K, benefits relevant results that are retrieved with higher relevance scores. Since it is usually calculated for multiple queries (Q), the mean value is obtained.

$$\text{MRR} = \frac{1}{Q} \sum_{q=1}^Q \frac{1}{\text{rank}_q} \quad (7)$$

- **Normalized Discounted Cumulative Gain (nDCG) [66]** is another well-known order-aware IR metric composed of some simpler metrics. Firstly, Cumulative Gain (CG@K) computes the sum of the relevance scores of the first K retrieved results.

$$\text{CG@K} = \sum_{k=1}^K \text{relevance}_k \quad (8)$$

However, since CG@K is not order-aware, the formula for Discounted Cumulative Gain (DCG@K) adds a logarithmic penalty to address this issue.

$$\text{DCG@K} = \sum_{k=1}^K \frac{\text{relevance}_k}{\log_2(1+k)}$$

Finally, the normalized Discounted Cumulative Gain (nDCG@K) normalizes the DCG@K value by considering the Ideal DCG@K (iDCG@K), which represents the maximum possible value of DCG@K. Therefore,

$$\text{nDCG@K} = \frac{\text{DCG@K}}{\text{iDCG@K}} \quad (9)$$

3.4.2. Evaluation using synthetically generated questions

On the other hand, a common evaluation approach for RAG systems [61,67] is applied for completing the evaluation. Using the Llama 3 70B model by Meta [30], a question-answering dataset is synthetically generated. For this purpose, each validated article text (>1 sentence) from the Spanish MIRACL Wikipedia dump is split using the Langchain Recursive text splitter with the configuration shown in Table 3. After splitting the article into different parts, the Llama 3 model generates a question-answer pair based on the text chunk. To ensure data quality, each generated sample is filtered, and only samples in which the answer is literally contained within the source chunk text are retained. This process yields a dataset comprising chunks, questions, and answers derived from all validated articles in the Spanish MIRACL Wikipedia dump. Similarly, the same process has been applied to a subset of the English MIRACL Wikipedia dump.

Using the previously created datasets, the evaluation is outlined as follows: (1) First, each question is queried against the vector database. (2) Second, a score is assigned according to Eq. (5), considering two factors: whether the answer is contained in the retrieved chunks, and the rank of the chunk containing the answer. For this purpose, the content of each chunk must be checked to determine if it contains the expected answer. If no chunk contains the answer, the score is set to zero. Conversely, if the answer is literally contained, then the rank of the retrieved chunk is considered. Since vector databases return chunks ordered by similarity score, the chunk with the highest similarity score should ideally comprise the answer. However, this is not always the case, and a lower-ranked chunk may contain the answer. In such cases, the evaluation score should be lower than in cases where this chunk is retrieved in the first place. Consequently, Score@K returns a value of one if the first-ranked chunk ($z = 1$) contains the answer and decreases its value as the position of the chunk containing the answer decreases in the ranking.

The computation of Score@K, formally defined in Eq. (10), operates on the set C , which contains the K chunks retrieved from the vector database. Each chunk and the answer are ASCII character sequences. Therefore, an answer is considered contained in a chunk z only if the same sequence of characters is found in the chunk. In such cases, a score ranging from one to zero is given. Retrieval order is considered by assigning higher values when z is small.

$$A_{ASCII} = \{char(x) \mid x \in \mathbb{N}, 0 \leq x \leq 127\}$$

$$C = \bigcup_{z=1}^K chunk_z$$

$$\begin{aligned} answer &= x_1 x_2 x_3 \dots x_k & x_i &\in A_{ASCII} & \forall i &\in \{1, 2, \dots, k\} \\ chunk_z &= y_{z,1} y_{z,2} y_{z,3} \dots y_{z,m} & y_{z,j} &\in A_{ASCII} & \forall j &\in \{1, 2, \dots, m\} \end{aligned}$$

$$answer \subseteq chunk_z \Leftrightarrow \exists i \in \{1, 2, \dots, m - k + 1\}, \forall j \in \{1, 2, \dots, k\}, y_{i+j-1} = x_j$$

$$Score@K = \begin{cases} \frac{K - z + 1}{K} & \text{if } \exists z \in \{1, 2, \dots, K\} \mid answer \subseteq chunk_z \\ 0 & \text{if } \forall z \in \{1, 2, \dots, K\} \mid answer \not\subseteq chunk_z \end{cases} \quad (10)$$

In summary, two types of evaluations have been conducted. The first one uses MIRACL passage labels to compute the novel metrics introduced in this study, as well as well-known IR metrics. This enables a

comprehensive assessment of retrieval performance. The second evaluation employs synthetically generated questions to compute the Score@K metric, which provides insight into the effectiveness of retrieval under different queries.

4. Results

This section presents the results of both evaluation strategies. First, the chunk length statistics are obtained for each of the four strategies applied. This analysis is of paramount importance because the length of the chunks may affect the results obtained for the different metrics. Later, the results obtained for SRA@K, SNRA@K, RCC@K, NRCC@K, and the most relevant IR metrics are exposed. Finally, the Score@K obtained through evaluation with synthetically generated questions is presented.

4.1. Chunk length analysis

As illustrated in Table 4, statistics have been calculated for the words and sentences within each of the chunking strategies applied to the Spanish MIRACL Wikipedia dump articles. These statistics include the mean, standard deviation, median, and the maximum, and minimum number of sentences and words. Analogously, Table 5 presents the statistics obtained with each of the chunking strategies applied on the article subset from the English MIRACL Wikipedia dump.

As expected, Tables 4 and 5 support the hypothesis that Kamradt's strategy yields longer chunks, as clearly states the significantly higher 75th percentile value compared to our strategies. Therefore, not only are longer chunks obtained, but the IR metrics will also be biased when evaluated because Kamradt's chunks are larger and have a higher probability of finding a MIRACL passage contained in them than in a smaller chunk.

To mitigate this bias, the same evaluation has been conducted by truncating the chunk content to the first 512 words. This length has been chosen because popular embedding models have very restrictive input sequence lengths. For example, the input for *bilingual-embedding-large* [45] is limited to 512 tokens. Since truncating by the number of tokens varies depending on the tokenizer, it has been decided to work with the first 512 words of each chunk. This approach ensures that extensive chunks do not compromise the integrity of the metrics obtained. Moreover, chunks whose relevant information is located at the beginning will be substantially rewarded. Consequently, the metrics obtained following this approach will be more reliable, focusing on the core requirements of an effective RAG retrieval phase and minimizing potentially noisy text that could negatively impact the generation phase.

Please note that the last row of Table 5 includes chunk statistics for a static splitter, configured with a line break (\n) as the delimiter. This splitter has been included as a baseline to compare static splitters versus semantic ones. For the English subset, all metrics will be also computed for this splitter to enhance the comparability of our results with different baselines.

In summary, the aforementioned IR metrics have been computed for both the full-length (non-limited) and 512-word truncated chunks. This provides a foundation for an in-depth study of the results, being able to determine which of the splitting strategies will result in a better performance in the RAG architecture.

4.2. Retrieval performance obtained for chunks from both strategies using MIRACL queries and labelled passages

Table 6 presents the results obtained for the IR retrieval metrics stated in Section 3.4.1 for the articles from the Spanish MIRACL Wikipedia dump. Each metric is computed using Kamradt's splitting strategy and for each of the three configurations set for the strategy proposed in this article.

The Wilcoxon signed-rank test has been employed as a non-

Table 3

Recursive character text splitter parameters.

Parameter	Value
Chunk Size	128
Chunk Overlap	20
Separators	., !, ?, \n, \r\n, \n\n
Length Function	len(nltk.tokenize.word_tokenize)

Table 4

Chunk Length Statistics from different splitting strategies using Spanish MIRACL. First row corresponds to Kamradt's splitting strategy using a window size of three sentences ($w = 3$). Following rows stand for our strategy with the three different configurations of initial size (n) and regular size (m).

Chunking Strategy		Mean	Std	Median	Max	Min	25th Percentile	75th Percentile
Kamradt $w = 3$	Words	213.20	285.00	114.00	27,173	0	54	261
	Sentences	8.53	9.64	5.00	1035	1	4	12
$n = 4$	Words	87.77	45.87	83.00	4999	0	57	112
	Sentences	4.95	1.35	5.00	417	1	5	5
$n = 6$	Words	124.44	64.60	121.00	5142	0	82	161
	Sentences	6.63	2.06	7.00	414	1	7	7
$n = 8$	Words	156.56	84.74	156.00	5129	0	97	208
	Sentences	8.11	2.85	9.00	417	1	7	9

Table 5

Chunk Length Statistics from different splitting strategies using English MIRACL subset. First three rows correspond to Kamradt's splitting strategy using a window size of two, three and four sentences ($w = 2, w = 3, w = 4$), respectively. Following rows stand for our strategy with the three different configurations of initial size (n) and regular size (m). Final row represents the static splitting strategy using |n as delimiter.

Chunking Strategy		Mean	Std	Median	Max	Min	25th Percentile	75th Percentile
Kamradt $w = 2$	Words	192.58	251.851	101.0	7535	0	43.0	245.0
	Sentences	10.64	11.41	7.0	320	1	4.0	13.0
Kamradt $w = 3$	Words	198.01	255.07	107.0	9705	0	47.0	251.0
	Sentences	10.92	11.48	7.0	363	1	4.0	14.0
Kamradt $w = 4$	Words	202.72	257.34	114.0	8664	0	50.0	255.0
	Sentences	11.15	11.56	7.0	364	1	4.0	14.0
$n = 4$	Words	75.78	37.70	72.0	2712	0	51.0	95.0
	Sentences	5.04	1.42	5.0	319	1	5.0	5.0
$n = 6$	Words	107.73	52.93	106.0	3388	0	74.0	137.0
	Sentences	6.80	2.16	7.0	363	1	7.0	7.0
$n = 8$	Words	136.51	69.33	137.0	3685	0	91.0	178.0
	Sentences	8.37	2.98	9.0	319	1	8.0	9.0
Static Splitter(\n)	Words	49.92	55.52	33.0	2065	0	5.0	75.0
	Sentences	3.49	2.51	3.0	216	1	2.0	4.0

Table 6

IR metrics results for entire chunks (left) and chunks truncated to 512 words (right) from Spanish articles. Best values are highlighted in bold. K. w3 column stands for Kamradt's strategy with a window size of 3 (w3). Columns with nX - mY indicate the results obtained for our strategy with an initial size (n) of X sentences and a regular size (m) of Y sentences.

	Full-length Chunks				512 Word-Limited Chunks			
	K. w3	n4 - m2	n6 - m3	n8 - m4	K. w3	n4 - m2	n6 - m3	n8 - m4
hit_rate@10	0.6793	0.51426	0.61974	0.65681	0.4850	0.3810	0.4597	0.5029
precision@10	0.1604	0.08168	0.11368	0.13118	0.0852	0.0545	0.0732	0.0845
recall@10	0.3451	0.18783	0.25771	0.29718	0.1817	0.1239	0.1613	0.1883
F1@10	0.2021	0.10466	0.145	0.16709	0.1071	0.0695	0.0926	0.1073
MRR@10	0.4933	0.31896	0.4112	0.44785	0.3287	0.2248	0.2820	0.3214
NDCG@10	0.3409	0.1812	0.24859	0.2836	0.1895	0.1204	0.1569	0.1841
SRA@10	11,823	19,962	18,665	17,787	17,231	22,695	22,714	22,416
SNRA@10	7747	11,715	11,303	11,017	10,222	12,587	12,733	12,682
RCC@10	16,968	8937	12,565	14,521	9274	5997	8150	9317
NRCC@10	6957	3137	4771	5740	3884	2188	3289	3939
SRA@K + RCC@10	28,791	28,899	31,230	32,308	26,505	28,692	30,864	31,733
SNRA@K + NRCC@10	14,704	14,852	16,074	16,757	14,106	14,775	16,022	16,621

parametric alternative to the paired Student's *t*-test. This choice is justified by the nature of the data, which did not meet the normality assumptions required for parametric testing. The Wilcoxon test is

appropriate for analysing matched samples, as in our case, since the same query provides results for both strategies. Tables 7 and 8 present the significance test results obtained for metrics computed using non-

Table 7

Wilcoxon significance test results for metrics obtained using non-limited chunks. Tests are performed between Kamradt's approach with window size of three ($w = 3$) and each of our configurations for Spanish articles. P values lower than 0.05 are highlighted in bold, denoting statistical significance.

K. w3	n4-m2		n6-m3		n8-m4	
	W	P_value	W	P_value	W	P_value
hit_rate@10	58,446.00	<0.001	58,322.50	<0.001	61,074.00	0.0202
precision@10	130,906.00	<0.001	233,303.00	<0.001	322,448.50	<0.001
recall@10	159,548.00	<0.001	252,957.00	<0.001	345,816.00	<0.001
F1@10	127,935.00	<0.001	229,438.50	<0.001	322,173.50	<0.001
MRR@10	221,420.00	<0.001	319,525.00	<0.001	361,047.00	<0.001
NDCG@10	258,440.00	<0.001	483,728.00	<0.001	678,380.50	<0.001

Table 8

Wilcoxon significance test results for metrics obtained using 512 word-limited chunks. Tests are performed between Kamradt's approach with window size of three ($w = 3$) and each of our configurations for Spanish articles. P values lower than 0.05 are highlighted in bold, denoting statistical significance.

K. w3	n4-m2		n6-m3		n8-m4	
	W	P_value	W	P_value	W	P_value
hit_rate@10	55,419.00	<0.001	60,605.00	0.004	52,216.00	0.0075
precision@10	109,207.00	<0.001	174,688.00	<0.001	215,314.50	0.2164
recall@10	118,041.50	<0.001	179,164.00	<0.001	209,229.50	0.0555
F1@10	108,694.50	<0.001	174,291.50	<0.001	211,923.50	0.1108
MRR@10	172,654.50	<0.001	244,832.50	<0.001	269,732.50	0.2299
NDCG@10	211,924.50	<0.001	353,043.50	<0.001	469,560.50	0.5774

limited and 512 word-limited chunks, respectively.

Regarding the results of full-length chunks, the splitting strategy proposed by Kamradt has been shown to outperform all configurations of our proposed strategy. Statistical significance tests further support this finding, revealing p-values below 0.05 across all configurations. However, when considering the SRA@K, SNRA@K, RCC@K, and NRCC@K metrics, as well as their compounds, the strategy proposed in this paper is distinguished by its ability to identify a large number of chunks originating from the same article or included in the target chunk (SRA@K + RCC@K). Nevertheless, it also retrieves a higher number of chunks from non-relevant articles or contained in non-relevant chunks (SNRA@K + NRCC@K).

A more dispersed scenario is found between both strategies and configurations when considering the 512-word truncated chunks. Kamradt's proposal achieves better results in metrics such as precision@K, MRR@K, and NDCG@K. Conversely, our approach, configured with an initial size of eight and a regular size of four, performs better in terms of hit rate, recall, and F1 score. However, statistical significance tests only support this finding for the hit rate metric. The metrics proposed in this paper yield analogous results, with our strategy significantly gathering more chunks from relevant articles or contained in positive labelled chunks (SRA@K + RCC@K), while Kamradt's approach retrieves a bit less number of chunks from negative labelled articles and chunks (SNRA@K + NRCC@K).

Relevant findings stand out when shifting towards results computed on the article subset from the English MIRACL Wikipedia dump (Tables 9 and 10). Regardless of word limitation, (1) the three sentence-sized window is shown to be Kamradt's optimal configuration, and (2) the performance of static splitters is far from that of any well-configured semantic approach.

As shown in Table 9, the results obtained with Spanish articles are aligned with those from full-length chunks. Kamradt's approach outperforms all configurations of our proposed strategy. The statistical significance tests in Table 11 further support these findings, denoting statistical significance for most results. However, our method can identify more chunks from relevant articles (SRA@K + RCC@K) and

non-relevant (SNRA@K + NRCC@K) articles.

As shown in Table 10, clearer results are obtained with English articles when working with 512 word-limited chunks. Using an initial size of eight and a regular size of four, our approach noticeably surpasses any of Kamradt's splitting configurations on traditional IR metrics. In this case, statistical significance tests only denote significance for the hit rate metric when using an initial size of eight and a regular size of four. Furthermore, the tendency to retrieve a higher number of relevant (SRA@K + RCC@K) and non-relevant (SNRA@K + NRCC@K) articles is preserved.

Concerning the static splitter results, both the full-length and word-limited settings perform worse than the three sentence window-sized Kamradt approach and our n8-m4 configuration. Furthermore, statistical tests underscore the significance of these results for all IR metrics (see Table 12).

4.3. Retrieval performance obtained for chunks from both strategies using synthetically generated questions

Since a value for Score@K is obtained whenever a question is queried against the vector database, the total sum (Total score@10 sum), average (Total score@10 avg) and standard deviation (Total score@10 std) of these values have been computed. Table 13 shows the results obtained for Score@10 and the retrieval performance of each strategy using full-length and 512 word-limited chunks. As before, the results obtained for the Spanish MIRACL Wikipedia dump are shown for Kamradt's strategy with a window size of three. For our strategy, the three configurations previously mentioned are evaluated. Next, results are computed for Kamradt's strategy with window sizes of two, three, and four sentences, as well as for the three configurations of our strategy and for the static splitter, using the English MIRACL Wikipedia dump.

As demonstrated in Table 13, the strategy outlined in this paper performs remarkably well in comparison to Kamradt's strategy. Certainly, regardless of the length of the chunks, the $n = 6$; $m = 3$ and $n = 8$; $m = 4$ configurations demonstrate a substantial improvement and significantly outperform Kamradt's strategy. The results obtained for the

Table 9

IR metrics results for entire chunks from English articles. Best values are highlighted in bold. K. wZ columns stand for Kamradt's strategy with a window size of Z (wZ). Columns with nX – mY indicate the results obtained for our strategy with an initial size (n) of X sentences and a regular size (m) of Y sentences.

	Full-length chunks						
	K. w2	K. w3	K. w4	n4 – m2	n6 – m3	n8 – m4	Static splitter
hit_rate@10	0.5671	0.5671	0.5572	0.2636	0.4029	0.5273	0.4228
precision@10	0.0746	0.0761	0.0741	0.0293	0.0452	0.0616	0.0497
recall@10	0.2844	0.2869	0.2772	0.1114	0.1697	0.2450	0.1947
F1@10	0.1062	0.1088	0.1059	0.0425	0.0646	0.0894	0.0714
MRR@10	0.3886	0.3915	0.3993	0.1740	0.2933	0.3627	0.2737
NDCG@10	0.2644	0.2739	0.2713	0.1096	0.1749	0.2337	0.1763
SRA@10	395	413	415	690	638	602	321
SNRA@10	161	219	219	335	333	307	190
RCC@10	196	164	164	64	99	134	109
NRCC@10	133	141	131	38	55	73	52
SRA@K + RCC@10	591	577	579	754	737	736	430
SNRA@K + NRCC@10	294	360	350	373	388	380	242

Table 10

IR metrics results for chunks truncated to 512 words from English articles. Best values are highlighted in bold. K. wZ column stand for Kamradt's strategy with a window size of Z (wZ). Columns with nX – mY indicate the results obtained for our strategy with an initial size (n) of X sentences and a regular size (m) of Y sentences.

	512 Word-Limited chunks						
	K. w2	K. w3	K. w4	n4 – m2	n6 – m3	n8 – m4	Static splitter
hit_rate@10	0.2786	0.2885	0.2786	0.1890	0.2537	0.3432	0.2238
precision@10	0.0318	0.0333	0.0333	0.0203	0.0273	0.0383	0.0248
recall@10	0.1418	0.1440	0.1408	0.0814	0.1037	0.1566	0.0954
F1@10	0.0466	0.0488	0.0483	0.0294	0.0383	0.0549	0.0350
MRR@10	0.2062	0.2038	0.2112	0.1218	0.1775	0.2372	0.1285
NDCG@10	0.1388	0.1414	0.1431	0.0801	0.1064	0.1533	0.0838
SRA@10	478	489	495	710	681	651	334
SNRA@10	261	274	282	350	359	338	178
RCC@10	68	71	76	45	59	84	54
NRCC@10	51	53	51	22	29	40	24
SRA@K + RCC@10	546	560	571	755	740	735	388
SNRA@K + NRCC@10	312	327	333	372	388	378	202

Table 11

Wilcoxon significance test results for metrics obtained using 512 word-limited (left) and non-limited chunks (right). Test are performed between Kamradt's approach with window size of three ($w = 3$) and each of our configurations for English articles. P values lower than 0.05 are highlighted in bold, denoting statistical significance.

K. w3	512 Word-Limited Chunks						Full-length chunks					
	n4-m2		n6-m3		n8-m4		n4-m2		n6-m3		n8-m4	
	W	P_value	W	P_value	W	P_value	W	P_value	W	P_value	W	P_value
hit_rate@10	77.5	<0.001	252.0	0.2367	139.5	0.0285	1050.0	<0.001	480.0	<0.001	780.0	0.3621
precision@10	126.0	<0.001	334.5	0.1703	222.0	0.0509	1113.0	<0.001	517.5	<0.001	945.0	0.0151
recall@10	108.5	<0.001	254.5	0.0223	260.0	0.2504	1117.0	<0.001	402.5	<0.001	978.0	0.0400
F1@10	106.5	<0.001	266.5	0.0333	264.0	0.2775	1074.5	<0.001	371.0	<0.001	929.0	0.0202
MRR@10	214.5	<0.001	462.0	0.1942	617.0	0.1981	1641.5	0.0034	1150.0	<0.001	2403.5	0.3640
NDCG@10	238.0	<0.001	464.5	0.0263	715.5	0.2783	1656.0	0.0012	1063.5	<0.001	2494.0	0.0957

Table 12

Wilcoxon significance test results for metrics obtained using 512 word-limited (left) and non-limited chunks (right). Test are performed between the static splitter with '\n' as delimiter and each of our configurations for English articles. P values lower than 0.05 are highlighted in bold, denoting statistical significance.

Static splitter	512 Word-Limited Chunks						Full-length chunks					
	n4-m2		n6-m3		n8-m4		n4-m2		n6-m3		n8-m4	
	W	P_value	W	P_value	W	P_value	W	P_value	W	P_value	W	P_value
hit_rate@10	480.0	0.3072	348.5	0.3428	168.0	<0.001	1050.0	<0.001	1404.0	0.5688	900.0	0.0126
precision@10	494.5	0.1892	380.5	0.4667	226.5	<0.001	1113.0	<0.001	1790.5	0.4402	1317.0	0.0333
recall@10	516.0	0.3353	403.0	0.7210	196.0	<0.001	1117.0	<0.001	1659.0	0.2126	1300.5	0.0440
F1@10	508.5	0.2991	407.0	0.7602	197.0	<0.001	1074.5	<0.001	1691.0	0.2657	1286.5	0.0377
MRR@10	756.5	0.7336	612.0	0.0581	394.0	<0.001	1641.5	0.0034	2468.0	0.7140	1992.0	0.0109
NDCG@10	753.5	0.4293	814.0	0.1841	429.0	<0.001	1656.0	0.0012	2862.0	0.8039	2243.5	0.0051

Table 13

Mean score@10 results for Spanish articles (best values are highlighted in bold).

		Total score@10 sum	Total score@10 avg	Total score@10 std
Full-length Chunks	Kamradt w = 3	1072,687.9	0.7678	0.3915
	n = 4	1036,846.1	0.7422	0.4058
	m = 2			
	n = 6	1101,198.1	0.7882	0.3777
	m = 3			
512 Word-Limited Chunks	n = 8	1128,901.0	0.8081	0.3639
	m = 4			
	Kamradt w = 3	1072,161.4	0.7674	0.3916
	n = 4	1038,877.9	0.7436	0.4050
	m = 2			
	n = 6	1101,651.0	0.7885	0.3774
	m = 3			
	n = 8	1129,416.6	0.8084	0.3636
	m = 4			

total score@10 sum and total score@10 average show that our chunking strategy leads to a better retrieval, where the sought chunk is retrieved more often and better ordered within the overall retrieved chunks than when using Kamradt's chunking approach. Moreover, our strategies have a smaller total score@10 standard deviation, providing even more reliability to our results. This is further endorsed by the significance tests

Table 14

Wilcoxon significance test results for Score@10 metrics between Kamradt's approach and each of our configurations for Spanish articles using 512 word-limited (left) and non-limited chunks (right). P values lower than 0.05 are highlighted in bold, denoting statistical significance.

K. w3	512 Word-Limited Chunks		Full-Length Chunks	
	W	p_value	W	p_value
n = 4	40,666,912,205.00	0.00	40,443,833,642.50	0.00
m = 2				
n = 6	38,388,765,400.50	0.00	38,442,115,610.50	0.00
m = 3				
n = 8	31,678,347,703.00	0.00	31,478,039,517.00	0.00
m = 4				

shown in Table 14.

Analogous results are obtained for the articles from the English MIRACL Wikipedia dump in Table 15. In both the $n = 6$; $m = 3$ and $n = 8$; $m = 4$ configurations, a better score@10 sum and average is yielded, outperforming the static splitter, which obtains worse results, as well as any of Kamradt's configurations. Furthermore, the score@10 standard deviation remains smaller in our approach, which lends reliability to our results. Table 16 demonstrates the significance of our results using significance tests. The Wilcoxon test has been computed between each configuration of our strategy and Kamradt's approach with a window size of four sentences, as it is the configuration with better score@10.

5. Discussion

Enhancing retrieval performance is a critical task within any RAG pipeline framework. Deciding how to split the text – or whether to split it at all – significantly impacts subsequent information retrieval. The present study proposes a new semantic splitting technique that overcomes the limitations of Kamradt's strategy and improves the information retrieval phase of a RAG pipeline. Moreover, a rigorous evaluation framework has been proposed and used to determine the impact of splitting strategies on retrieval performance.

Conventional text-splitting approaches often rely on methods that are insensitive to semantic structure, such as character chunking or recursive character splitting. This potentially can result in the creation of text chunks that lack semantic unity and fail to encapsulate a single core concept. As Wu et al. [68] state, maintaining semantic integrity and ensuring the semantic independence of retrieved passages are critical principles for high-quality RAG architectures. Therefore, relying on these semantically agnostic approaches within a RAG pipeline is a significant limitation. The semantic splitting technique proposed in this study aims to mitigate this weakness by applying a semantic chunking strategy that considers text semantics and overcomes limitations of previous semantic approaches. Kamradt introduced semantic chunking as a promising method for enhancing retrieval performance with his semantic text splitting technique. However, a significant limitation of

this technique was identified. Including sentences in chunks without considering the meaning of the entire chunk results in a semantically limited text-splitting strategy. Therefore, there is a risk of splitting text into long, non-semantically meaningful chunks due to gradual shifts in meaning. Consequently, the chunks computed using Kamradt's technique may contain irrelevant information, thus leaving the potential of semantic chunking untapped. In contrast, the proposed methodology leverages a fully semantic approach to text splitting, evaluating the semantic coherence of the entire chunk. This nuanced distinction is critical because the presence of irrelevant information in chunks can introduce noise during the generation phase, which may potentially increase hallucinations [6].

The initial hypothesis that Kamradt's strategy results in a longer chunk length is finally substantiated by the statistics shown in Tables 4 and 5. Undoubtedly, the chunks obtained using Kamradt's strategy are longer than those obtained using any of our strategy's configurations. Due to the nature of the evaluation process, however, this will significantly bias the results, since there is a greater probability of MIRACL passages being contained within longer chunks. Consequently, evaluation results obtained for 512 word-limited chunks are most suitable for comparing the performance of both strategies.

To validate the cross-lingual application of the proposed strategy, both Spanish and English Wikipedia articles are used for evaluation. Using Spanish articles, the strategy proposed in this study stands out for its ability to retrieve a higher number of relevant chunks. The Hit rate@K, Recall@K, and F1@K metrics prove this behaviour. Likewise, SRA@K and RCC@K also support the efficacy of our technique by illustrating the retrieval of a higher number of same relevant articles and relevant chunks contained in their corresponding MIRACL article. However, Kamradt's approach performs slightly better for metrics that consider the order of the retrieved documents. The values obtained for the MRR@K and NDCG@K metrics demonstrate this behaviour. Nonetheless, retrieving as many relevant articles as possible is considered more important than ordering them. This is because, later, in a real RAG pipeline, different reranking approaches can be applied. Therefore, if Kamradt's strategy retrieves fewer relevant articles in a better order, performance might be worse, and further improvements, like reranking, would be less effective [69,70].

For English articles, the results demonstrate a consistently higher performance of our approach, despite a general decrease in performance for this language. This degradation may be due to the multilanguage embedding model's inability to fully capture English-specific semantics. Nevertheless, our approach yields demonstrably improved IR metrics compared to Kamradt's optimal configuration, while the static splitter's performance is significantly worse. The SRA@K + RCC@K metric achieves its highest values with $n = 8$; $m = 4$, although the $n = 4$; $m = 2$ configuration yields a higher SRA@K at the expense of RCC@K. Since maximizing the number of relevant chunks contained (RCC@K) is considered more significant and challenging, we identify the $n = 8$; $m = 4$ configuration as superior. Therefore, analysis of IR metrics across both languages reveals that our approach clearly outperforms the established baselines.

Moreover, the evaluation using synthetic questions exhibits clear results regarding the enhancement achieved by our strategy for both English (see Table 13) and Spanish (see Table 15) Wikipedia articles. The splitting strategy proposed in this paper clearly outperforms both Kamradt's strategy and the static splitter. It is imperative to acknowledge that the metric employed for this evaluation (Score@10) is also an order-aware metric. Therefore, these results may differ from the previous ones (see Table 6), as the MRR@K and NDCG@K metrics show slightly superior results for Kamradt's approach in Spanish. However, this discrepancy is attributed to how each metric accentuates the order penalty. As demonstrated in Fig. 5, the value of each metric (y-axis) is plotted against the rank (x-axis) at which the chunk is retrieved. While MRR@K and iDCG@K impose significant penalties on chunks from second to seventh, owing to their logarithmic nature, Score@K adopts a

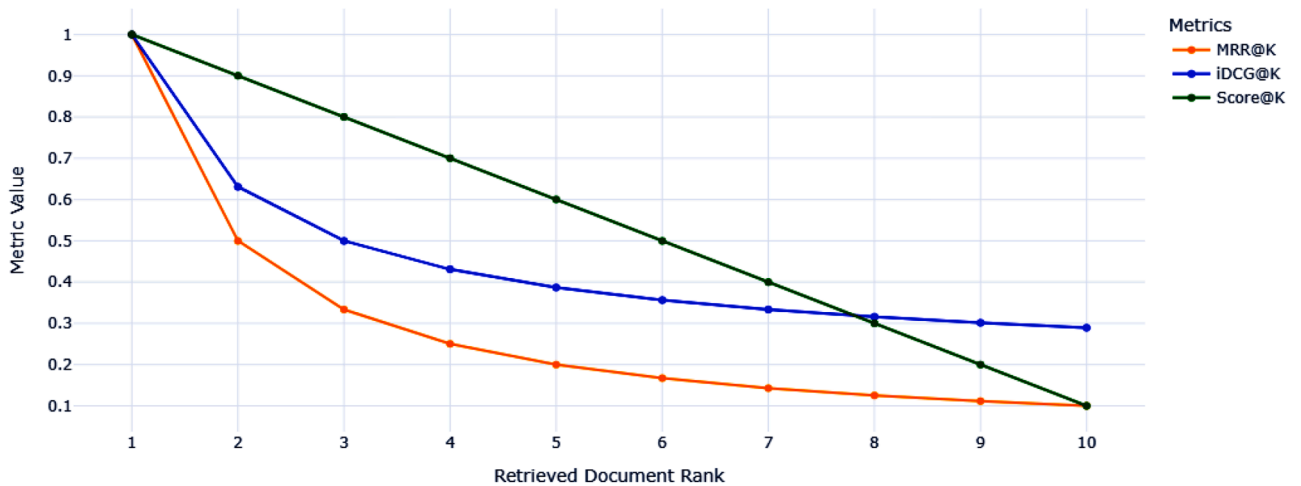
Table 15
Mean score@10 results for English articles (best values are highlighted in bold).

		Total score@10 sum	Total score@10 avg	Total score@10 std
Full-length Chunks	Kamradt $w = 2$	116,157.8	0.7941	0.3608
	Kamradt $w = 3$	118,093.4	0.8073	0.3531
	Kamradt $w = 4$	120,849.4	0.8262	0.3382
	$n = 4$	115,811.35	0.7917	0.3630
	$m = 2$	121,309.92	0.8293	0.3390
	$n = 6$	121,309.92	0.8293	0.3390
	$m = 3$	122,932.75	0.8404	0.3321
	$n = 8$	122,932.75	0.8404	0.3321
	$m = 4$	122,932.75	0.8404	0.3321
512 Word- Limited Chunks	Static splitter	81,795.43	0.5592	0.4576
	Kamradt $w = 2$	116,810.10	0.7985	0.3567
	Kamradt $w = 3$	119,230.12	0.8151	0.3459
	Kamradt $w = 4$	120,935.72	0.8268	0.3378
	$n = 4$	115,501.91	0.7896	0.3648
	$m = 2$	121,245.76	0.8289	0.3394
	$n = 6$	121,245.76	0.8289	0.3394
	$m = 3$	122,747.61	0.8391	0.3334
	$n = 8$	122,747.61	0.8391	0.3334
	$m = 4$	122,747.61	0.8391	0.3334
	Static splitter	81,260.89	0.5555	0.4587

Table 16

Wilcoxon significance test results for Score@10 metrics obtained using non-limited and 512 word-limited chunks between Kamradt's strategy with window size of four sentences (left), and Wilcoxon significance test results for Score@10 metrics obtained for English articles using non-limited and 512 word-limited chunks between Kamradt's strategy and the static splitter (right). P values lower than 0.05 are highlighted in bold, denoting statistical significance.

	K. w4				Static splitter			
	512 Word-Limited Chunks		Full-Length Chunks		512 Word-Limited Chunks		Full-Length Chunks	
	W	p_value	W	p_value	W	p_value	W	p_value
$n = 4$	246,635,446.5	0.00	249,771,766.0	0.00	298,548,676.0	0.00	288,085,928.5	0.00
$m = 2$								
$n = 6$	195,366,256.0	0.00	194,281,068.0	<0.001	219,424,592.0	0.00	219,597,374.0	0.00
$m = 3$								
$n = 8$	153,193,779.5	<0.001	149,247,366.5	<0.001	209,784,162.0	0.00	205,896,564.5	0.00
$m = 4$								

MRR@K, iDCG@K and Score@K Functions**Fig. 5.** MRR@K, iDCG@K and Score@K functions.

linear tendency, imposing a considerably smaller penalty on chunks not retrieved in the first position. Nonetheless, it is important to note that Score@K also penalises relevant chunks retrieved from the eighth position onwards compared to iDCG@K. Consequently, this signifies that our chunking strategy sways retrieval by fetching a higher number of sought chunks within the first positions of the ranking. Conversely, chunks built using Kamradt's approach are more likely to be retrieved in the first position; however, a smaller amount of the desired chunks are retrieved since their semantic significance is lower due to the aforementioned limitations.

A thorough statistical analysis of the Score@K metric reveals a significant improvement over the method initially proposed by Kamradt. For the Spanish data, the proposed approach yielded scores of 0.8081 and 0.8084 for full-length and 512 word-limited chunks, respectively. In comparison, Kamradt's method achieved scores of 0.7678 and 0.7674. In English, the proposed approach resulted in scores of 0.8404 and 0.8391 for full-length and 512 word-limited chunks, respectively, while Kamradt's strategy obtained scores of 0.8262 and 0.8268. This improvement results in a 4 % and 2 % increase in the rate of relevant information supplied to the generation phase for Spanish and English, respectively. These findings suggest that the proposed strategy, regardless of the language, increases the probability of providing the generative model with pertinent information, thereby improving the overall RAG pipeline performance.

Finally, regarding the various configurations applied for Kamradt's strategy, a window size of three sentences has been demonstrated to yield more competitive results for IR metrics. However, for the Score@K metric, a window size of four sentences attains superior performance. In

the context of our study, the $n = 8$ and $m = 4$ configuration yields better results. However, this is simply because these chunks are larger and the probability of containing MIRACL passages is higher. Therefore, it is important to highlight that, in a real RAG pipeline, the configuration of these parameters will rely on domain-specific needs such as the maximum context length or the expected answer length.

6. Conclusions

Since the inception of Retrieval Augmented Generation, research has focused on improving the reliability of chatbots based on LLMs. Despite the use of diverse models, prompts, and strategies to enhance RAG performance, there is still a huge lack of knowledge on how the text splitting process impacts retrieval performance. This research study analyses state-of-the-art text splitting techniques within the RAG framework. Specially, semantical splitting techniques have been examined, identifying their current limitations and proposing a novel semantical chunking technique.

The proposed strategy successfully addresses limitations from state-of-the-art approaches and enhances retrieval performance by improving the cohesion of the split information, thereby enhancing RAG's performance in the retrieval phase. This milestone has been grounded by the exhaustive evaluation methodology applied in this research. Firstly, our evaluation approach addresses the issue of assessing the implication of the splitting process in the retrieval performance, applying the labels given in an IR dataset for custom chunks. Consequently, an innovative approach has been proposed for leveraging MIRACL labelled passages to obtain well-known IR metrics as well as newly defined ones. Secondly,

the evaluation process is concluded by defining a new metric for measuring the retrieval performance by means of synthetic generated questions.

The results of this research have proved that Kamradt's semantic splitting process results in the obtention of longer chunks that seem to lose significant semantic meaning. This phenomenon is evident in the performance of IR metrics where our strategy outperforms Kamradt's one when 512 word-limited chunks are used, as well as with the results obtained for Score@K with synthetic questions. Furthermore, due to the nature of the different order-aware metrics, it can be stated that, when using our splitting approach, the desired chunks are more likely to be retrieved, even if they are not retrieved in the first positions of the score ranking.

Moreover, evaluation using synthetically generated questions clearly demonstrate the efficacy of the proposed approach for improving RAG pipeline performance. The analysis reveals a quantifiable superiority of our approach in Score@K, which leads to a 4 % and a 2 % increase in answer delivery to the generation phase in Spanish and English, respectively, when compared to Kamradt's strategy.

As a result, the proposed chunking strategy offers a pathway to more reliable RAG systems, demonstrating enhanced performance compared to state-of-the-art approaches and exhibiting strong cross-domain and cross-lingual generalizability. By mitigating the generation of inaccurate or hallucinatory responses, this strategy has been demonstrated to improve performance in knowledge-intensive applications. This enhanced reliability is of particular importance in domains where information accuracy is critical, such as healthcare, legal analysis, or aviation maintenance. The broader impact of this work lies in its potential to foster greater trust and utility in RAG-based artificial intelligence, thereby enabling its effective deployment in high-stakes decision-making contexts.

Finally, research conducted for the definition of a coherent and solid evaluation framework has revealed a strong bias produced by the length of the chunks. Given the substantial discrepancy in the lengths of Kamradt's chunks relative to those obtained through our strategy, any comparisons drawn between them are inherently imbalanced and potentially misleading. Consequently, when comparing two different chunking strategies, it is imperative to be mindful of the chunk lengths in order to ensure a fair and consistent evaluation.

7. Limitations and future work

Even though promising results were exposed, some limitations about our study have been identified. First and foremost, this chunking methodology does not allow a 'dynamic' chunking threshold, as Kamradt's approach does by using the percentile operation. In our methodology, since the decisions for splitting the text are taken directly while running through the text, it is not possible to apply this operation. Therefore, searching for the optimal threshold used in our strategy might result in a more difficult process, as it might depend on the nature of the text and the embedding model employed.

On the other hand, while the configuration of initial (n) and regular (m) sizes is beneficial for enhancing and optimizing the splitting process, it is necessary to examine which configuration might work best for each specific domain and problem.

Future work will focus on tackling the abovementioned limitations. On the one hand, research about how to automatically determine the optimal threshold might be done. On the other hand, research regarding methods to automatically determine the chunking parameters will be of utmost importance to improve even more the retrieval effectiveness of RAG pipelines.

Data availability

The MIRACL Wikipedia dump used in this research along with many other dumps in different languages are available at the MIRACL GitHub

repository (<https://github.com/project-miracle/miracle>). Specifically, the Spanish and English dumps used in this study can be found at the following links: <https://archive.org/download/eswiki-20220301/eswiki-20220301-pages-articles-multistream.xml.bz2>, <https://archive.org/download/enwiki-20190201/enwiki-20190201-pages-articles-multistream.xml.bz2>, respectively.

CRediT authorship contribution statement

Antonio Moreno-Cediel: Writing – original draft, Software, Methodology, Investigation, Conceptualization. **Eva Garcia-Lopez:** Writing – original draft, Supervision, Project administration, Investigation, Funding acquisition. **Antonio Garcia-Cabot:** Writing – original draft, Supervision, Resources, Project administration, Investigation, Funding acquisition. **David De-Fitero-Dominguez:** Writing – review & editing, Methodology, Investigation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was supported by the project “Tecnologías Inteligentes para la Fabricación, el diseño y las Operaciones en entornos iNdustriales” (TIFON, PLEC2023-010251) through the call Proyectos de I + D + i en líneas estratégicas - Transmisiones 2023.

References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, (2023). <https://doi.org/10.48550/arXiv.1706.03762>.
- [2] E.Y. Zhang, A.D. Cheok, Z. Pan, J. Cai, Y. Yan, From Turing to transformers: a comprehensive review and tutorial on the evolution and applications of generative transformer models, *Science* 5 (2023) 46, <https://doi.org/10.3390/sci5040046>.
- [3] I. Akermi, J. Heinecke, F. Herledan, Transformer based natural language generation for question-answering, in: B. Davis, Y. Graham, J. Kelleher, Y. Sripada (Eds.), *Proc. 13th Int. Conf. Nat. Lang. Gener. Association for Computational Linguistics*, Dublin, Ireland, 2020, pp. 349–359, <https://doi.org/10.18653/v1/2020.inlg-1.41>.
- [4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, D. Kiela, Retrieval-augmented generation for knowledge-intensive NLP tasks, (2021). <https://doi.org/10.48550/arXiv.2005.11401>.
- [5] Y. Zhang, Y. Li, L. Cui, D. Cai, L. Liu, T. Fu, X. Huang, E. Zhao, Y. Zhang, Y. Chen, L. Wang, A.T. Luu, W. Bi, F. Shi, S. Shi, Siren's song in the AI ocean: a survey on hallucination in large language models, (2023). <https://doi.org/10.48550/arXiv.2309.01219>.
- [6] F. Shi, X. Chen, K. Misra, N. Scales, D. Dohan, E. Chi, N. Schärli, D. Zhou, Large language models can be easily distracted by irrelevant context, (2023). <https://doi.org/10.48550/arXiv.2302.00093>.
- [7] O. Yoran, T. Wolfson, O. Ram, J. Berant, Making retrieval-augmented language models robust to irrelevant context, (2024). <https://doi.org/10.48550/arXiv.2310.01558>.
- [8] S. Setty, K. Jijo, E. Chung, N. Vidra, Improving retrieval for RAG based question answering models on financial documents, (2024). <https://doi.org/10.48550/arXiv.2404.07221>.
- [9] RetrievalTutorials/tutorials/LevelsOfTextSplitting/5 Levels Of Text Splitting. Ipyynb at main · FullStackRetrieval-com/RetrievalTutorials, GitHub (n.d.). <https://github.com/FullStackRetrieval-com/RetrievalTutorials/blob/main/tutorials/LevelsofTextSplitting/5 Levels Of Text Splitting.ipynb> (accessed September 3, 2024).
- [10] P. Verma, S2 Chunking: a hybrid framework for document segmentation through integrated spatial and semantic analysis, (2025). <https://doi.org/10.48550/arXiv.2501.05485>.
- [11] X. Zhang, N. Thakur, O. Ogundepo, E. Kamalloo, D. Alfonso-Hermelo, X. Li, Q. Liu, M. Rezagholizadeh, J. Lin, Making a MIRACL: multilingual information retrieval across a continuum of languages, (2022). <https://doi.org/10.48550/arXiv.2210.09984>.
- [12] G.G. Chowdhury, *Introduction to Modern Information Retrieval*, Facet publishing, 2010.
- [13] G. Chowdhury, Introduction to modern information retrieval. http://www.lapwing.org.uk/cgi-bin/miva?lap/merchant.mv+Screen=PROD&Store_Code=1&Product_Code=489, 2004 accessed October 17, 2024.

- [14] M. Taube, C.D. Gull, I.S. Wachtel, Unit terms in coordinate indexing, *Am. Doc* 3 (1952) 213. Pre-1986.
- [15] G. Salton, Automatic Text processing: the transformation, analysis, and Retrieval of Information By Computer, Addison-Wesley Longman Publishing Co., Inc., USA, 1989.
- [16] H.P. Luhn, A statistical approach to mechanized encoding and searching of literary information, *IBM J. Res. Dev.* 1 (1957) 309–317, <https://doi.org/10.1147/rd.14.0309>.
- [17] K. Sparck Jones, A statistical interpretation of term specificity and its application in retrieval, *J. Doc.* 28 (1972) 11–21, <https://doi.org/10.1108/eb026526>.
- [18] S.E. Robertson, K.S. Jones, Relevance weighting of search terms, *J. Am. Soc. Inf. Sci.* 27 (1976). <https://www.proquest.com/docview/1301247214/citation/F28CDD11919C45EEEPQ/1>. accessed October 21, 2024.
- [19] S. Robertson, H. Zaragoza, The Probabilistic Relevance Framework: BM25 and beyond, *Found. Trends® Inf. Retr.* 3 (2009) 333–389, <https://doi.org/10.1561/15000000019>.
- [20] G. Salton, Developments in automatic text retrieval, *Science* 253 (1991) 974–980.
- [21] G. Salton, C. Buckley, Flexible Text Matching For Information Retrieval, Cornell University, 1990. <https://hdl.handle.net/1813/6998>. accessed October 21, 2024.
- [22] G. Salton, C. Buckley, Global text matching for information retrieval, *Science* 253 (1991) 1012–1015.
- [23] T. Mikolov, K. Chen, G. Corrado, J. Dean, Efficient estimation of word representations in vector space, (2013). <https://doi.org/10.48550/arXiv.1301.3781>.
- [24] J. Pennington, R. Socher, C. Manning, GloVe: global vectors for word representation, in: A. Moschitti, B. Pang, W. Daelemans (Eds.), *Proc. 2014 Conf. Empir. Methods Nat. Lang. Process. EMNLP*, Association for Computational Linguistics, Doha, Qatar, 2014, pp. 1532–1543, <https://doi.org/10.3115/v1/D14-1162>.
- [25] M.T. Pilehvar, J. Camacho-Collados, *Embeddings in Natural Language processing: Theory and Advances in Vector Representations of Meaning*, Morgan & Claypool Publishers, 2020.
- [26] O. Melamud, J. Goldberger, I. Dagan, context2vec: learning generic context embedding with bidirectional LSTM, in: S. Riezler, Y. Goldberg (Eds.), *Proc. 20th SIGNLL Conf. Comput. Nat. Lang. Learn.*, Association for Computational Linguistics, Berlin, Germany, 2016, pp. 51–61, <https://doi.org/10.18653/v1/K16-1006>.
- [27] M.E. Peters, M. Neumann, M. Iyyer, M. Gardner, C. Clark, K. Lee, L. Zettlemoyer, Deep contextualized word representations, (2018). <https://doi.org/10.48550/arXiv.1802.05365>.
- [28] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, BERT: pre-training of deep bidirectional transformers for language understanding, (2019). <https://doi.org/10.48550/arXiv.1810.04805>.
- [29] N. Reimers, I. Gurevych, Sentence-BERT: sentence embeddings using siamese BERT-networks, (2019). <https://doi.org/10.48550/arXiv.1908.10084>.
- [30] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan, A. Goyal, A. Hartshorn, A. Yang, A. Mitra, A. Srivankumar, A. Korenev, A. Hinsvark, A. Rao, A. Zhang, A. Rodriguez, A. Gregerson, A. Spataru, B. Roziere, B. Biron, B. Tang, B. Chern, C. Caucheteux, C. Nayak, C. Bi, C. Marra, C. McConnell, C. Keller, C. Touret, C. Wu, C. Wong, C.C. Ferrer, C. Nikolaidis, D. Allonsius, D. Song, D. Pintz, D. Livshits, D. Esiobu, D. Choudhary, D. Mahajan, D. Garcia-Olano, D. Perino, D. Hupkes, E. Lakomkin, E. AlBadawy, E. Lobanova, E. Dinan, E.M. Smith, F. Radenovic, F. Zhang, G. Synnaeve, G. Lee, G.L. Anderson, G. Nail, G. Mialon, G. Pang, G. Cucurell, H. Nguyen, H. Korevaar, H. Xu, H. Touvron, I. Zarov, I.A. Ibarra, I. Kloumann, I. Misra, I. Evtimov, J. Copet, J. Lee, J. Geffert, J. Vranes, J. Park, J. Mahadeokar, J. Shah, J. van der Linde, J. Billock, J. Hong, J. Lee, J. Fu, J. Chi, J. Huang, J. Liu, J. Wang, J. Yu, J. Bitton, J. Spisak, J. Park, J. Rocca, J. Johnston, J. Saxe, J. Jia, K.V. Alwala, K. Upasani, K. Plawiak, K. Li, K. Heafield, K. Stone, K. El-Arini, K. Iyer, K. Malik, K. Chiu, K. Bhalla, L. Rantala-Young, L. van der Maaten, L. Chen, L. Tan, L. Jenkins, L. Martin, L. Madaan, L. Malo, L. Blecher, L. Landzaat, L. de Oliveira, M. Muzzi, M. Pasupuleti, M. Singh, M. Paluri, M. Kardas, M. Oldham, M. Rita, M. Pavlova, M. Kambadur, M. Lewis, M. Si, M.K. Singh, M. Hassan, N. Goyal, N. Torabi, N. Bashlykov, N. Bogoychev, N. Chatterji, O. Duchenne, O. Çelebi, P. Alrassy, P. Zhang, P. Li, P. Vasic, P. Wang, P. Bhargava, P. Dubal, P. Krishnan, P.S. Koura, P. Xu, Q. He, Q. Dong, R. Srinivasan, R. Ganapathy, R. Calderer, R.S. Cabral, R. Stojnic, R. Raileanu, R. Girdhar, R. Patel, R. Sauvestre, R. Polidoro, R. Sumbaly, R. Taylor, R. Silva, R. Hou, R. Wang, S. Hosseini, S. Chennabasappa, S. Singh, S. Bell, S.S. Kim, S. Edunov, S. Nie, S. Narang, S. Raparthy, S. Shen, S. Wan, S. Bhosale, S. Zhang, S. Vandenheide, S. Batra, S. Whitman, S. Sootla, S. Collet, S. Gururangan, S. Borodinsky, T. Herman, T. Fowler, T. Sheasha, T. Georgiou, T. Scialom, T. Speckbacher, T. Mihaylov, T. Xiao, U. Karn, V. Goswami, V. Gupta, V. Ramanathan, V. Kerkez, V. Gonguet, V. Do, V. Vogeti, V. Petrovic, W. Chu, W. Xiong, W. Fu, W. Meers, X. Martinet, X. Wang, X.E. Tan, X. Xie, X. Jia, X. Wang, Y. Goldschlag, Y. Gaur, Y. Babaei, Y. Wen, Y. Song, Y. Zhang, Y. Li, Y. Mao, Z.D. Coudert, Z. Yan, Z. Chen, Z. Papakipos, A. Singh, A. Grattafiori, A. Jain, A. Kelsey, A. Shajnfeld, A. Gangidi, A. Victoria, A. Goldstand, A. Menon, A. Sharma, A. Boesenberg, A. Vaughan, A. Baevski, A. Feinstein, A. Kallet, A. Sangani, A. Yunus, A. Lupu, A. Alvarado, A. Caples, A. Gu, A. Ho, A. Poulton, A. Ryan, A. Ramchandani, A. Franco, A. Saraf, A. Chowdhury, A. Gabriel, A. Bharambe, A. Eisenstein, A. Yazdan, B. James, B. Maurer, B. Leonhardi, B. Huang, B. Loyd, B.D. Paola, B. Paranjape, B. Liu, B. Wu, B. Ni, B. Hancock, B. Wasti, B. Spence, B. Stojkovic, B. Gamido, B. Montalvo, C. Parker, C. Burton, C. Mejia, C. Wang, C. Kim, C. Zhou, C. Hu, C.-H. Chu, C. Cai, C. Tindal, C. Feichtenhofer, D. Civin, D. Beatty, D. Kreymer, D. Li, D. Wyatt, D. Adkins, D. Xu, D. Testuggine, D. David, D. Parikh, D. Liskovich, D. Foss, D. Wang, D. Le, D. Holland, E. Dowling, E. Jamil, E. Montgomery, E. Presani, E. Hahn, E. Wood, E. Brinkman, E. Arcaute, E. Dunbar, E. Smothers, F. Sun, F. Kreuk, F. Tian, F. Ozgenel, F. Caggioni, F. Guzmán, F. Kanayet, F. Seide, G.M. Florez, G. Schwarz, G. Badeer, G. Sweet, G. Halpern, G. Thattai, G. Herman, G. Sizov, Guangyi, Z., G. Lakshminarayanan, H. Shojanazeri, H. Zou, H. Wang, H. Zha, H. Habeeb, H. Rudolph, H. Suk, H. Aspegren, H. Goldman, I. Damla, I. Molybog, I. Tufanov, I.-E. Veliche, I. Gat, J. Weissman, J. Geboski, J. Kohli, J. Asher, J.-B. Gaya, J. Marcus, J. Tang, J. Chan, J. Zhen, J. Reizenstein, J. Teboul, J. Zhong, J. Jin, J. Yang, J. Cummings, J. Carvill, J. Shepard, J. McPhie, J. Torres, J. Ginsburg, J. Wang, K. Wu, KH U., K. Saxena, K. Prasad, K. Khandelwal, K. Zand, K. Matosich, K. Veeraraghavan, K. Michelen, K. Li, K. Huang, K. Chawla, K. Lakhotia, K. Huang, L. Chen, L. Garg, L. A., L. Silva, L. Bell, L. Zhang, L. Guo, L. Yu, L. Moshkovich, L. Wehrstedt, M. Khabisa, M. Avalani, M. Bhatt, M. Tsimipoukelli, M. Mankus, M. Hasson, M. Lennie, M. Reso, M. Groshev, M. Naumov, M. Lathi, M. Keneally, M.L. Seltzer, M. Valko, M. Restrepo, M. Patel, M. Vyatskov, M. Samvelyan, M. Clark, M. Macey, M. Wang, M.J. Hermoso, M. Metanat, M. Rastegari, M. Bansal, N. Santhanam, N. Parks, N. White, N. Bawa, N. Singhal, N. Egebo, N. Usunier, N.P. Laptev, N. Dong, N. Zhang, N. Cheng, O. Chernoguz, O. Hart, O. Salpekar, O. Kalinli, P. Kent, P. Parekh, P. Saab, P. Balaji, P. Rittner, P. Bontrager, P. Roux, P. Dolla, P. Zvyagina, P. Ratanchandani, P. Yuvraj, Q. Liang, R. Alao, R. Rodriguez, R. Ayub, R. Murthy, R. Nayani, R. Mitra, R. Li, R. Hogan, R. Battey, R. Wang, R. Maheswari, R. Howes, R. Rinott, S.J. Bondu, S. Datta, S. Chugh, S. Hunt, S. Dhillion, S. Sidorov, S. Pan, S. Verma, S. Yamamoto, S. Ramaswamy, S. Lindsay, S. Lindsay, S. Feng, S. Lin, S.C. Zha, S. Shankar, S. Zhang, S. Zhang, S. Wang, S. Agarwal, S. Sajuyigbe, S. Chintala, S. Max, S. Chen, S. Kehoe, S. Satterfield, S. Govindaprasad, S. Gupta, S. Cho, S. Virk, S. Subramanian, S. Choudhury, S. Goldman, T. Remez, T. Glaser, T. Best, T. Kohler, T. Robinson, T. Li, T. Zhang, T. Matthews, T. Chou, T. Shaked, V. Vontimitta, V. Ajayi, V. Montanez, V. Mohan, V.S. Kumar, V. Mangla, V. Albiero, V. Ionescu, V. Poenaru, V.T. Mihailescu, V. Ivanov, W. Li, W. Wang, W. Jiang, W. Bouaziz, W. Constable, X. Tang, X. Wang, X. Wu, X. Wang, X. Xia, X. Wu, X. Gao, Y. Chen, Y. Hu, Y. Jia, Y. Qi, Y. Li, Y. Zhang, Y. Zhang, Y. Adi, Y. Nam, Yu, W., Y. Hao, Y. Qian, Y. He, Z. Rait, Z. DeVito, Z. Rosnbrick, Z. Wen, Z. Yang, Z. Zhao, The Llama 3 Herd of models, (2024). <https://doi.org/10.48550/arXiv.2407.21783>.
- [31] A. Pal, L.K. Umapathi, M. Sankararubbu, Med-HALT: medical domain hallucination test for large language models, (2023). <https://doi.org/10.48550/arXiv.2307.15343>.
- [32] Z. Zhang, C. Wang, Y. Wang, E. Shi, Y. Ma, W. Zhong, J. Chen, M. Mao, Z. Zheng, LLM hallucinations in practical code generation: phenomena, mechanism, and mitigation, *Proc. ACM Softw. Eng.* 2 (2025), <https://doi.org/10.1145/3728894>. ISSTA022:481-ISSTA022:503.
- [33] A Survey on Hallucination in Large Language Models: Principles, Taxonomy, challenges, and open questions | ACM transactions on information systems, (n.d.). <https://dl.acm.org/doi/full/10.1145/3703155> (accessed September 15, 2025).
- [34] Z. Gekhman, G. Yona, R. Aharoni, M. Eyal, A. Feder, R. Reichart, J. Herzig, Does fine-tuning LLMs on new knowledge encourage hallucinations? in: Y. Al-Onaizan, M. Bansal, Y.-N. Chen (Eds.), *Proc. 2024 Conf. Empir. Methods Nat. Lang. Process Association for Computational Linguistics*, Miami, Florida, USA, 2024, pp. 7765–7784, <https://doi.org/10.18653/v1/2024.emnlp-main.444>.
- [35] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, H. Wang, Retrieval-augmented generation for large language models: a survey, (2024). <https://doi.org/10.48550/arXiv.2312.10997>.
- [36] T.B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D.M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, D. Amodei, Language models are few-shot learners, (2020). <https://doi.org/10.48550/arXiv.2005.14165>.
- [37] K. Luo, Z. Liu, S. Xiao, K. Liu, BGE landmark embedding: a chunking-free embedding method for retrieval augmented long-context large language models, (2024). <https://doi.org/10.48550/arXiv.2402.11573>.
- [38] S. Chen, S. Wong, L. Chen, Y. Tian, Extending context window of large language models via positional interpolation, (2023). <https://doi.org/10.48550/arXiv.2306.15595>.
- [39] Z. Zhong, H. Liu, X. Cui, X. Zhang, Z. Qin, Mix-of-granularity: optimize the chunking granularity for retrieval-augmented generation, (2024). <https://doi.org/10.48550/arXiv.2406.00456>.
- [40] A.J. Yepes, Y. You, J. Milczek, S. Laverde, R. Li, Financial report chunking for effective retrieval augmented generation, (2024). <https://doi.org/10.48550/arXiv.2402.05131>.
- [41] N. Muennighoff, N. Tazi, L. Magne, N. Reimers, MTEB: massive text embedding benchmark, (2023). <https://doi.org/10.48550/arXiv.2210.07316>.
- [42] S. Sturua, I. Mohr, M.K. Akram, M. Günther, B. Wang, M. Krimmel, F. Wang, G. Mastrapas, A. Koukounas, N. Wang, H. Xiao, jina-embeddings-v3: multilingual embeddings with Task LoRA, (2024). <https://doi.org/10.48550/arXiv.2409.10173>.
- [43] C. Lee, R. Roy, M. Xu, J. Raiman, M. Shoyebi, B. Catanzaro, W. Ping, NV-Embed: improved techniques for training LLMs as generalist embedding models, (2024). <https://doi.org/10.48550/arXiv.2405.17428>.
- [44] dunzhang/stella_en_1.5B_v5 · Hugging Face, n.d. https://huggingface.co/dunzhang/stella_en_1.5B_v5, 2024. accessed October 28
- [45] Lajavaness/bilingual-embedding-large · Hugging Face, n.d. <https://huggingface.co/Lajavaness/bilingual-embedding-large>, 2024. accessed November 22
- [46] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, A. Levskaya, J. Heek, K. Xiao, S. Agrawal, J. Dean, Efficiently scaling transformer inference, (2022). <https://doi.org/10.48550/arXiv.2211.05102>.

- [47] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C.H. Yu, J. Gonzalez, H. Zhang, I. Stoica, Efficient memory management for large language model serving with paged attention, in: Proc. 29th Symp. Oper. Syst. Princ., Association for Computing Machinery, New York, NY, USA, 2023, pp. 611–626, <https://doi.org/10.1145/3600066.3613165>.
- [48] K. Guu, K. Lee, Z. Tung, P. Pasupat, M. Chang, Retrieval augmented language model pre-training, in: Proc. 37th Int. Conf. Mach. Learn., PMLR, 2020, pp. 3929–3938, in: <https://proceedings.mlr.press/v119/guu20a.html>, accessed July 19, 2024.
- [49] S. Borgeaud, A. Mensch, J. Hoffmann, T. Cai, E. Rutherford, K. Millican, G. van den Driessche, J.-B. Lespiau, B. Damoc, A. Clark, D. de L. Casas, A. Guy, J. Menick, R. Ring, T. Hennigan, S. Huang, L. Maggiore, C. Jones, A. Cassirer, A. Brock, M. Paganini, G. Irving, O. Vinyals, S. Osindero, K. Simonyan, J.W. Rae, E. Elsen, L. Sifre, Improving language models by retrieving from trillions of tokens, (2022). <https://doi.org/10.48550/arXiv.2112.04426>.
- [50] O. Rubin, J. Berant, Retrieval-pretrained transformer: long-range language modeling with self-retrieval, (2024). <https://doi.org/10.48550/arXiv.2306.13421>.
- [51] LangChain, n.d. <https://www.langchain.com/>, 2024, accessed July 24
- [52] LlamaIndex, Data framework for LLM applications, n.d. <https://www.llamaindex.ai/>, 2024, accessed July 24
- [53] langchain_text_splitters.base.TextSplitter — LangChain 0.2.10, (n.d.). https://api.python.langchain.com/en/latest/base/langchain_text_splitters.base.TextSplitter.html#langchain_text_splitters.base.TextSplitter (accessed July 24, 2024).
- [54] langchain_text_splitters.character.CharacterTextSplitter — LangChain 0.2.10, (n.d.). https://api.python.langchain.com/en/latest/character/langchain_text_splitters.character.CharacterTextSplitter.html#langchain_text_splitters.character.CharacterTextSplitter (accessed July 24, 2024).
- [55] langchain_text_splitters.character.RecursiveCharacterTextSplitter — LangChain 0.2.10, (n.d.). https://api.python.langchain.com/en/latest/character/langchain_text_splitters.character.RecursiveCharacterTextSplitter.html#langchain_text_splitters.character.RecursiveCharacterTextSplitter (accessed July 24, 2024).
- [56] Split by HTML section | LangChain, (n.d.). https://python.langchain.com/v0.1/docs/modules/data_connection/document_transformers/HTML_section_aware_splitter/ (accessed September 3, 2024).
- [57] langchain_text_splitters.markdown.MarkdownTextSplitter — LangChain 0.2.10, (n.d.). https://api.python.langchain.com/en/latest/markdown/langchain_text_splitters.markdown.MarkdownTextSplitter.html#langchain_text_splitters.markdown.MarkdownTextSplitter (accessed July 24, 2024).
- [58] langchain_text_splitters.python.PythonCodeTextSplitter — LangChain 0.2.10, (n.d.). https://api.python.langchain.com/en/latest/python/langchain_text_splitters.python.PythonCodeTextSplitter.html#langchain_text_splitters.python.PythonCodeTextSplitter (accessed July 24, 2024).
- [59] M.H. Hadid, Z.T. Al-Qaysi, Q.M. Hussein, R.A. Aljanabi, I.R. Abdulqader, M. S. Suzani, W.L. Shir, Semantic image retrieval analysis based on deep learning and singular value decomposition, Appl. Data Sci. Anal. 2024 (2024) 17–31, <https://doi.org/10.58496/ADSA/2024/003>.
- [60] M. Sallam, M.A. Shnan, Enhancing semantic image retrieval using self-supervised learning: a label-efficient approach, Babylon. J. Mach. Learn. 2025 (2025) 42–60, <https://doi.org/10.58496/BJML/2025/004>.
- [61] S. Sivasothy, S. Barnett, S. Kurniawan, Z. Rasool, R. Vasa, RAGProbe: an automated approach for evaluating RAG applications, (2024). <https://doi.org/10.48550/arXiv.2409.19019>.
- [62] G. Attardi, WikiExtractor, GitHub repos. <https://github.com/attardi/wikiextractor>, 2015.
- [63] M. Sanderson, Test collection based evaluation of information retrieval systems, Found. Trends® Inf. Retr. 4 (2010) 247–375, <https://doi.org/10.1561/1500000009>.
- [64] R.K. Hamad, Integrating machine learning and genetic algorithms to enhance gene-disease classification: an XBNet-based framework, Babylon. J. Mach. Learn. 2025 (2025) 1–12, <https://doi.org/10.58496/BJML/2025/001>.
- [65] E. Bassani, AmenRa/ranx. <https://github.com/AmenRa/ranx>, 2024 accessed November 4, 2024.
- [66] K. Järvelin, J. Kekäläinen, Cumulated gain-based evaluation of IR techniques, ACM Trans. Inf. Syst. 20 (2002) 422–446, <https://doi.org/10.1145/582415.582418>.
- [67] Z. Rackauckas, A. Câmara, J. Zavrel, Evaluating RAG-Fusion with RAGElo: an automated Elo-based framework, (2024). <https://doi.org/10.48550/arXiv.2406.14783>.
- [68] S. Wu, Y. Xiong, Y. Cui, H. Wu, C. Chen, Y. Yuan, L. Huang, X. Liu, T.-W. Kuo, N. Guan, C.J. Xue, Retrieval-augmented generation for natural language processing: a survey, (2025). <https://doi.org/10.48550/arXiv.2407.13193>.
- [69] T. Niu, S. Joty, Y. Liu, C. Xiong, Y. Zhou, S. Yavuz, JudgeRank: leveraging large language models for reasoning-intensive reranking, (2024). <https://doi.org/10.48550/arXiv.2411.00142>.
- [70] M. Mukherjee, S. Kim, X. Chen, D. Luo, T. Yu, T. Mai, From documents to dialogue: building KG-RAG enhanced AI assistants, (2025). <https://doi.org/10.48550/arXiv.2502.15237>.